



北京工商大学

源代码级异步操作系统调试方法

团队成员：曾小红、王浩铭、武雪妍

指导教师：吴竞邦

计算机与人工智能学院

2026 年 8 月

目 录

1	项目背景	4
1.1	操作系统调试：特权级切换与符号表管理	4
1.2	Rust 异步程序调试：编译模型带来的执行流不透明	4
2	核心目标与总体方案	6
3	核心工作分述	9
3.1	异步跟踪开销问题：从静态预计算到运行时按需发现	9
3.1.1	问题的具体表现	9
3.1.2	运行时按需发现	10
3.1.3	具体实现	11
3.2	跟踪状态跨特权级保持问题：异步跟踪与 OS 调试的融合	15
3.2.1	问题的具体表现	15
3.2.2	解决方案：跨生命周期的保存与恢复机制	16
3.3	调试器的可视化问题：图形化界面设计	19
3.3.1	问题的具体表现	19
3.3.2	三层架构	20
3.3.3	界面布局与 workflow	21
3.3.4	物理流 + 逻辑流双轨协同	22
3.4	调试器可移植性问题：面向不同操作系统的通用化改进	23
3.4.1	问题的具体表现	23
3.4.2	改进一：用函数名指定特殊断点，摆脱对本地源码文件路径的依赖	24
3.4.3	改进二：放弃硬编码字段路径，改为依赖 GDB 提供的稳定接口读取变量	26
3.4.4	改进三：进程组管理从静态预配置到运行时动态自动注入	28
3.5	调试器的硬件适配问题：VisionFive 2 真机调试与线上化扩展	30
3.5.1	问题的具体表现	30
3.5.2	VisionFive 2 真机调试链路	30
3.5.3	内核态、用户态单步及跨特权级调试适配	32
3.5.4	线上硬件交互平台实现	32

目 录	3
3.5.5 线上平台与硬件调试能力结合	34
4 测试与验证	34
4.1 状态机单元测试与集成测试	34
4.1.1 OSSStateMachine 单元测试 (37 个)	34
4.1.2 MockMI2 集成测试 (4 个场景)	35
4.2 embassy 异步跟踪运行时无关性验证	35
4.2.1 测试设置	36
4.2.2 验证结果	36
4.2.3 验证结论	37
4.3 StarryOS 跨特权级调试验证	37
4.3.1 调试配置	37
4.3.2 调试流程验证	41
4.4 Async-os 统一调试场景验证	42
4.4.1 第一阶段: 内核态异步协程追踪 (已完成)	42
4.4.2 第二阶段: 跨特权级异步追踪 (进行中)	43
5 总结与展望	46
5.1 已完成工作总结	46
5.2 未来展望	47
6 项目分工	48

1 项目背景

1.1 操作系统调试：特权级切换与符号表管理

用 GDB 调试一个普通用户程序时，开发者只需要加载一次符号表¹，之后所有的断点设置和堆栈回溯都在同一个地址空间内完成。但调试操作系统内核时，情况完全不同：内核态和用户态使用不同的页表、不同的地址空间，更重要的是使用完全不同的符号表。当 CPU 在两种特权级²之间切换时，调试器必须同步切换符号表——否则断点将命中错误的地址，堆栈回溯也毫无意义。

如果由开发者手动完成这一过程，每次特权级切换都需要依次执行：卸载旧符号文件、加载新符号文件、逐一清除旧断点、逐一恢复新断点。在操作系统启动过程中，内核与用户态之间会发生数十次切换，手动操作根本不可行。

针对这一问题，已有工作实现了通过状态机³驱动的断点组⁴管理机制：调试器自动识别当前所在的特权级，在内核态与用户态的断点组之间自动切换，开发者只需在对应源码位置正常设置断点即可。

然而，这套机制在最初设计时依赖三个与操作系统源码组织方式相关的隐含假设：内核与用户程序源码在同一工作区内（可以通过文件路径区分断点归属）；所有系统调用经过用户态统一的入口（一个断点就能拦截全部切换）；调试开始前就知道会运行哪些用户程序（断点组可以提前写好配置）。这些假设在教学操作系统上成立，但推广到更广泛的操作系统时需要重新设计——这是本文要解决的核心问题之一。

1.2 Rust 异步程序调试：编译模型带来的执行流不透明

操作系统内核需要同时处理大量并发 I/O 请求（磁盘读写、网络收发等），传统做法是为每个请求创建一个线程。但每个线程都要占用独立的栈空间，线程之间的切换也需

¹符号表（symbol table）：编译器生成调试信息文件（如 ELF 中的 `.debug_info` 段），记录了函数名与内存地址的对应关系。GDB 利用它来处理断点设置和堆栈回溯。

²特权级（privilege level）：CPU 的运行级别。RISC-V 架构中，机器态（M-mode）权限最高，内核态（S-mode）次之，用户态（U-mode）最低。操作系统内核运行在高特权级，用户程序运行在低特权级。

³状态机（state machine）：一种通过定义状态和状态间转移规则来管理复杂流程的模型。在这里，状态机的四个状态分别对应内核态、从内核切换到用户态的过程中、用户态、从用户切换回内核态的过程中。

⁴断点组（breakpoint group）：每个地址空间维护一个独立的断点集合和对应的符号文件。调试器在切换地址空间时，自动清除当前组的断点、卸载当前组的符号文件，然后加载新组的符号文件、恢复新组的断点。

要保存和恢复大量寄存器，在资源受限的场景下代价很高。

Rust 的 `async/await` 机制提供了一种更轻量的方案。编译器将 `async` 函数编译为一个状态机，这个状态机不会主动执行——只有当外部调度器来驱动它时（这个操作叫 `poll`⁵），它才往前走一步，遇到 `.await` 就暂停，几乎不占用栈空间。这一特性使得用 Rust 编写异步操作系统内核成为趋势——已有项目明确提出利用 Rust 异步机制优化内核并发性能，将 I/O 操作封装为轻量级协程⁶，由内核调度器统一管理。

然而，这一编译模型也带来了调试层面的困难。编译器将 `async` 函数拆分为多个 `poll` 阶段后，函数的逻辑执行流被切分为离散的状态片段。每次 GDB 暂停时，它看到的物理调用栈⁷只能看到当前这一次 `poll` 的入口，无法呈现「这个 `Future`⁸在等待哪个 `Future`」「控制流是怎样经过多个 `poll` 周期走到这里的」。对于内核开发者而言，即使能在任一 `poll` 位置设置断点，也难以从栈帧信息中还原出完整的异步执行因果链。

针对这一问题，已有工作提出了关键观察：要解释一次 Rust 异步执行，仅恢复「谁在等待谁」（称为 `await edge`⁹）是不够的，还必须同时恢复「这次真实控制流是怎样推进到这里的」（称为 `call edge`¹⁰），两种边互补才能构成完整的异步执行因果图。

不过，已有工作在实现这一方法时采取的是「调试前全量插桩」策略：在调试开始前离线解析程序 DWARF 调试信息，预计算完整的依赖树，然后对依赖树中所有相关函数提前设置跟踪断点。这种做法的开销与程序规模成正比——对于一个包含上百个 `async` 函数的项目，每次启动都需要等待 DWARF 解析完成并设置数百个断点。此外，它通过 DWARF 中的类型偏移量来识别协程实例，无法区分同一异步函数的不同并发实例——

⁵`poll`: Rust 异步运行时的核心机制。每次 `poll` 推动 `Future` 状态机向前执行一步，直到在下一个 `.await` 处暂停，交出控制权。一个 `Future` 可能需要多次 `poll` 才能完成。

⁶协程 (coroutine): 可以暂停和恢复执行的函数。Rust 的 `async` 函数被编译为协程状态机，与线程不同，协程的切换不涉及操作系统内核，开销极低。

⁷物理调用栈 (physical call stack): GDB 实际展示的函数调用链，即传统调试器中的 Call Stack 面板。与之相对的「逻辑调用树」是我们还原出的 `async` 函数之间的等待依赖关系——反映的不是「谁调用了谁」，而是「谁在等待谁」。

⁸`Future`: Rust 异步编程的核心抽象，代表一个尚未完成的计算。`async` 函数编译后会产生一个实现了 `Future` 接口的状态机。

⁹`await edge` (等待边): 源码层面的异步等待关系——「这个 `Future` 在 `await` 哪个 `Future`」。通过读取编译器在状态机中自动生成的 `__awaitee` 字段来恢复。

¹⁰`call edge` (调用边): 机器层面的真实控制流转移——「谁通过函数调用驱动了这个 `Future` 的 `poll`」。因为 `await edge` 遇到同步包装函数（如 `block_on`）就会中断，需要通过动态设置断点来追踪跨越同步边界的控制流。

比如一个网络服务器同时处理十个连接，每个连接背后都是同一个 `async` 函数的不同实例，基于偏移量的识别方式会把它们混淆在一起。今年工作的一个重要方向就是用运行时按需发现替代调试前全量插桩，同时引入更精确的实例识别方法。

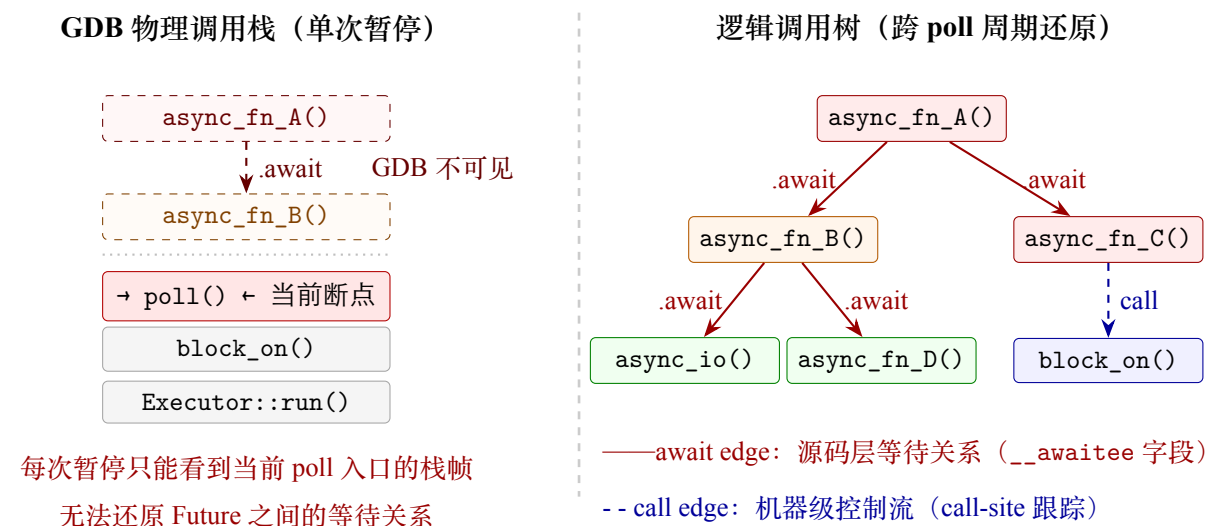


图 1: 物理调用栈与逻辑调用树的对比。左侧为 GDB 每次暂停时看到的调用栈——只能看到 `poll` 入口，看不到「谁在等待谁」。右侧为还原后的逻辑调用树——跨多个 `poll` 周期串联起完整的执行因果关系。

2 核心目标与总体方案

本项目的目标是将前序工作中验证的调试方法推广到更广泛的操作系统和真实异步项目上，并构建一个统一的调试平台。具体来说，我们面临四个核心问题：

1. **异步跟踪开销问题：**从静态预计算到运行时按需发现。已有的异步跟踪方法需要在调试前离线分析完整的程序调试信息、预计算依赖树并对所有相关函数设置跟踪断点（即插桩¹¹），开销与程序规模成正比。需要一种运行时按需发现的方式，只追踪当前执行路径上真正相关的异步函数。
2. **异步跟踪数据跨特权级保持问题：**异步跟踪与 **OS** 调试的融合。这是把异步调试和 **OS** 调试放在一起时才暴露的问题。操作系统在内核态和用户态之间切换时，需要

¹¹插桩（instrumentation）：在程序特定位置自动插入断点，以在运行时收集执行信息。这里指在 `Future` 的 `poll` 入口、内部函数调用点、返回点设置断点，以追踪异步执行流。

更换符号表和断点集——这会把异步跟踪积累的协程状态、追踪断点、白名单地址映射全部清空。需要一套保存与恢复机制：切换前把追踪状态打包保存，切换后按正确顺序逐一恢复。

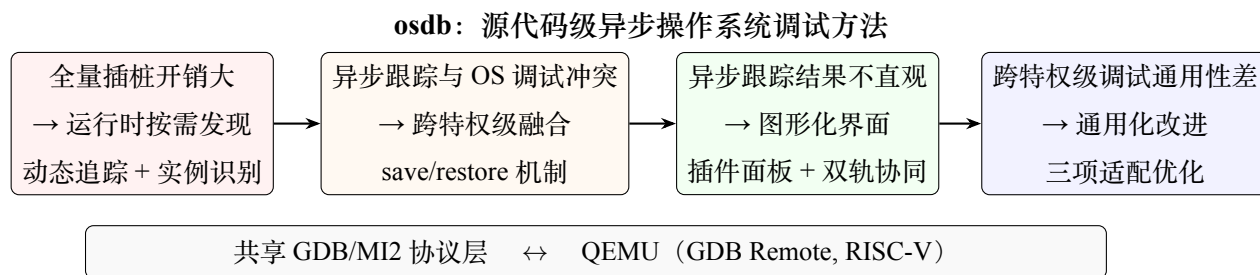
3. 调试器的可视化问题：图形化界面设计。异步跟踪工具最初只能在终端打印文本输出。对于包含几十个协程的实际程序，开发者很难从文字中看出「谁在等谁」「整个依赖树有多大」。更根本的是，VS Code 的调试界面只认识线程和栈帧，不理解异步程序的等待链。需要自建一个图形化面板，用树形图展示异步函数之间的等待和调用关系，用颜色区分协程和普通函数，点击节点就能跳转到源码。
4. 调试器可移植性问题：面向不同操作系统的通用化改进。已有的跨特权级调试机制依赖三个与操作系统源码组织方式相关的隐含假设，这些假设在组件化操作系统上不成立。需要让调试器摆脱这些假设，降低适配新操作系统的门槛。

核心问题	最终目标	技术项
全量插桩开销大	取消调试前静态分析，改为运行时按需发现	运行时按需发现机制
		Trace root 与白名单动态管理机制
异步跟踪与跨特权级调试冲突	异步跟踪与跨特权级调试融合，实现异步操作系统调试	OS 调试能力移植
		Save/Restore 跨特权级异步跟踪数据保持
异步跟踪结果不直观	图形化界面设计，跟踪结果可视化	实现 VS Code 插件化架构
		Async Inspector 图形化面板
跨特权级调试通用性差	针对组件化OS调试的通用化改进	函数名断点 + 通用收敛点 + 方向属性
		动态进程组自动注入
		跨 Rust 版本变量读取

图 2: 项目总述

技术项	完成 情况	说明
运行时按需发现机制	完成	去掉 DWARF 预解析，运行时读取 __awaiter 字段动态发现等待关系；以 (poll_sym, env_ptr) 二元组区分并发实例
Trace Root 与白名单动态管理	完成	引入 trace root 概念，支持手动指定和从 backtrace 自动推断；白名单按 crate 分组，支持运行时加载和热更新

技术项	完成 情况	说明
OS 调试能力移植	完成	将四状态机与断点组管理移植至 async-debug，编写 37 个单元测试和 4 个集成测试全部通过
Save/Restore 跟踪数据保持	完成	切换前保存四类数据（断点、协程状态、追踪记录、白名单），切换后五步有序恢复，保证跨地址空间协程追踪连续性
VS Code 插件化架构	完成	实现 Debug Adapter + Extension + Webview 三层架构；DebugAdapterTracker 拦截 stopped 事件驱动面板自动刷新
Async Inspector 图形化面板	完成	逻辑调用树实物/流双轨展示，标注 CID、poll 次数、状态；白名单 Gen→Apply→Trace 全流程集成于面板
函数名断点 + 收敛点 + 方向属性	完成	函数名突破外部 crate 限制；syscall 分发函数为通用收敛点；方向属性消除同地址空间方向歧义
动态进程组自动注入	完成	待注入队列保证任意时刻创建的进程组自动继承 user→kernel 边界断点，切换链条完整闭合
跨 Rust 版本变量读取	完成	GDB Pretty Printer → console 捕获 → 内存直读三步方案，不依赖 String 内部字段路径，兼容所有 Rust 版本
[验证]		
StarryOS 跨特权级调试验证	完成	在组件化 OS StarryOS 上完成全流程调试：内核初始化 → shell 启动 → 动态加载用户程序 → 断点组自动切换，全程无需手动干预
embassy 异步跟踪验证	完成	在无外部异步运行时的嵌入式框架 embassy 上成功追踪，验证了工具不依赖特定运行时内部数据结构的核心主张
Async-os 统一调试验证	部分完成	内核态异步协程追踪已完成（coroutine_test）；跨特权级异步追踪适配中（user_boot）



验证目标: StarryOS (组件化 OS) • embassy (嵌入式异步框架) • Async-os (异步操作系统)

图 3: 项目总览

3 核心工作分述

3.1 异步跟踪开销问题：从静态预计算到运行时按需发现

3.1.1 问题的具体表现

已有的异步跟踪方法采取「调试前全量插桩」的策略：（1）在调试开始前，离线解析目标程序 ELF 文件的 DWARF¹²调试信息，提取所有 poll 函数的返回类型和字段布局；（2）从每个 poll 函数的 `__awaitee` 字段类型推导出它可能等待的子 Future，递归展开形成完整的依赖树；（3）对依赖树中所有的 poll 入口函数设置跟踪断点。这意味着即使开发者只关心某一个具体的异步调用路径，系统也会对程序中所有可能的异步函数进行插桩。

这个策略有两个根本性问题。第一，插桩开销与程序规模成正比——对于一个包含上百个 async 函数的中型项目，每次启动调试都需要等待 DWARF 解析完成并设置数百个断点。第二，无法区分同一类型的不同实例——比如一个网络服务器同时处理十个连接，每个连接背后都是同一个 async 函数的不同实例，基于类型偏移量的识别方式会把它们混淆在一起。

¹²DWARF: ELF 文件中存储调试信息的标准格式，记录了变量类型、函数参数、结构体字段偏移等编译期信息。GDB 通过解析 DWARF 来处理栈帧回溯和变量查看。

维度	已有方法（静态预计算）	优化方向（运行时按需发现）
等待关系获取	调试前离线解析 DWARF，递归展开依赖树	运行时通过 GDB 读取 <code>__awaitee</code> 字段
插桩范围	依赖树中全部函数，全量提前设置	仅当前执行路径上真正相关的函数
协程识别	基于 DWARF 偏移量，无法区分实例	<code>(poll_symbol, env_ptr)</code> 二元组区分实例
追踪起点	没有明确概念，依赖全量插桩覆盖	引入 <code>trace root</code> ，自动推断或手动指定

3.1.2 运行时按需发现

改进的核心思路是：不再提前计算完整的依赖树，而是让调试器在程序实际执行到某个 `poll` 函数时，当场从内存中读取它的状态机字段，动态发现它在等待哪个子 `Future`，然后只对那个子 `Future` 设置跟踪断点。

等待关系的运行时获取。Rust 编译器在生成 `async` 状态机时，会在 `Future` 对象内部嵌入 `__awaitee`¹³ 字段——该字段的类型直接指向当前正在等待的子 `Future`。每次 GDB 在 `poll` 入口断点处暂停时，调试器通过 GDB 的类型系统读取当前 `Future` 的 `__awaitee` 字段，即可获知父 `Future` 在等待哪个子 `Future`，进而建立 `await edge`。这个过程完全在运行时完成，不需要任何预处理。

协程实例的精确识别。同一类型的多个并发实例共享相同的 `poll` 函数代码，仅靠函数名无法区分。改进使用 `(poll_symbol, env_ptr)` 二元组作为实例的唯一标识：`poll_symbol` 是 `poll` 函数的符号名（区分类型），`env_ptr` 是 `Future` 环境对象的地址（区分同一类型的不同实例）。每次 `poll` 事件发生时，系统以这个二元组查询影子栈¹⁴——若首次出现则分配新的协程 ID（`CID`¹⁵），若已存在则使用已有 `CID` 继续记录。

追踪根节点的引入。已有方法没有「追踪根节点」的概念——它通过白名单筛选出一批感兴趣的异步函数，然后对它们及其依赖树中的全部相关函数插桩。改进引入了明确的 `trace root` 概念：用户指定一个根节点函数（如 `ardb-trace <symbol>`），系统从该函数的 `__awaitee` 字段出发，顺藤摸瓜地自动发现被等待的子 `Future` 并安装跟踪断点——不再关心依赖树中有哪些不相关的函数。

¹³`__awaitee`: Rust 编译器在 `async` 状态机中自动生成的字段，其类型指向当前正在等待的子 `Future`。当 `Future` 在某个 `.await` 处暂停时，该字段记录了被等待的 `Future` 的类型信息。

¹⁴影子栈（shadow stack）：调试器在内存中维护的一个数据结构，记录每个协程实例的 `poll` 历史、等待关系、调用路径。与 GDB 的物理调用栈不同，它是跨 `poll` 周期累积的。

¹⁵`CID`（Coroutine ID）：全局唯一的协程标识符，与具体的 `Future` 实例绑定。同一类型的不同实例拥有不同的 `CID`，确保追踪数据不会混淆。

3.1.3 具体实现

运行时按需发现的核心逻辑由 GDB Python 脚本 (`runtime_trace.py`) 实现。当 GDB 在 `poll` 入口断点处停止时，脚本执行以下步骤：

```
1 def on_poll_entry(frame):
2     """每次进入 poll 函数时调用，按需发现等待关系"""
3     future_obj = read_current_future(frame)
4     cid = get_or_assign_cid(future_obj) # (poll_symbol, env_ptr) → CID
5
6     # 运行时读取 __awaitee 字段，发现被等待的子 Future
7     awaitee_type = extract_awaitee_type(future_obj)
8     if awaitee_type is not None:
9         # 只对当前正在等待的子 Future 设置跟踪断点
10        child_poll_fn = find_poll_function(awaitee_type)
11        if child_poll_fn and not already_tracking(child_poll_fn):
12            install_poll_entry_breakpoint(child_poll_fn)
13            record_await_edge(cid, child_poll_fn)
14
15    # 记录本次 poll 事件
16    record_poll_event(cid, frame)
```

代码段 1: 运行时读取 `__awaitee` 建立等待关系

下面分别展开等待关系的运行时获取、协程实例的精确识别、追踪根节点的引入三个子问题。

等待关系的运行时获取 已有方法需要在调试前离线解析 DWARF，从每个 `poll` 函数的返回类型中提取 `__awaitee` 字段的类型信息，递归展开完整的依赖树。改进后的方法不再做任何预处理——当程序执行到某个 `poll` 入口断点时，GDB Python 脚本直接从当前栈帧中读取 `Future` 对象的 `__awaitee` 字段。该字段的类型名直接指向被等待的子 `Future`，例如类型为 `async_fn_leaf` 表示父 `Future` 正在等待 `async_fn_leaf` 这个子 `Future`。脚本随即查找该子 `Future` 的 `poll` 函数符号，如果尚未被追踪，则在该 `poll` 函数入口安装跟踪断点。整个过程在程序实际执行到对应路径时才触发，依赖树中不相关的分支永远不会被插桩。

协程实例的精确识别 同一个 `async` 函数可能同时存在多个正在执行的实例——比如一个网络服务器对每个连接都创建一个异步任务，这些任务背后是同一份 `poll` 函数代码。调试器必须能区分「`async_handler` 的第 1 个实例」和「`async_handler` 的第 5 个实例」，否则它们的 `poll` 次数、等待关系、CID 会全部混淆。

已有方法通过 DWARF 中 `Future` 类型的偏移量来生成标识符——这只能区分不同的 `async` 函数类型，无法区分同一类型的不同实例。改进使用 `(poll_symbol, env_ptr)` 二元组作为协程实例的唯一标识：

- `poll_symbol` 是 `poll` 函数的符号名（如 `async_handler::{async_fn#0}::poll`），用于区分 `Future` 的类型——不同的 `async` 函数编译出不同的 `poll` 函数。
- `env_ptr` 是 `Future` 环境对象的地址。编译器为每个 `async` 函数实例在堆或栈上分配一个环境对象，存储状态机的当前状态、被等待的子 `Future` 引用、捕获的局部变量等。同一类型的不同实例拥有不同的环境对象地址。

每次 `poll` 事件发生时，系统以 `(poll_symbol, env_ptr)` 查询全局字典 `_CO_BY_KEY`。若命中，则返回已有的 CID，并累加该实例的 `poll` 次数；若未命中，则分配新的 CID，初始化 `poll` 次数为 1。

```

1 def get_or_assign_cid(future_obj):
2     """以 (poll_symbol, env_ptr) 为键，分配或查找 CID"""
3     poll_sym = str(future_obj.type)          # poll 函数符号名
4     env_ptr = int(future_obj.address)        # Future 环境对象地址
5     key = (poll_sym, env_ptr)
6
7     if key in _CO_BY_KEY:
8         # 已存在：复用 CID，累加 poll 计数
9         cid = _CO_BY_KEY[key]
10        _CO_META[cid]['poll_count'] += 1
11    else:
12        # 新实例：分配新 CID
13        _CO_NEXT_ID += 1
14        cid = _CO_NEXT_ID
15        _CO_BY_KEY[key] = cid
16        _CO_META[cid] = {

```

```
17         'poll_symbol': poll_sym,
18         'env_ptr': env_ptr,
19         'poll_count': 1,
20         'state': 'running',
21         'children': [],      # 被等待的子协程 CID 列表
22     }
23     return cid
```

代码段 2: 基于 (poll_symbol, env_ptr) 的实例识别与 CID 分配

CID 从 1 开始递增，全局唯一，与具体的 Future 实例绑定，贯穿整个调试会话。`_CO_META` 为每个 CID 维护 poll 历史、关联的子协程列表、当前状态（正在运行 / 等待中 / 已完成），这些元数据在 Async Inspector 面板中展示为树节点的注解信息。

追踪根节点的引入 前序方法没有「从哪个函数开始追踪」的概念——它通过白名单指定一批感兴趣的异步函数，然后对它们及其依赖树中的全部相关函数设置断点。这种方法的问题是：白名单中可能有几十个函数，但某次具体的调试会话只关心其中一条调用路径，其余路径上的断点全部是浪费。

改进引入了明确的 trace root¹⁶概念。用户通过 `ardb-trace <symbol>` 命令指定一个函数作为根节点。系统收到命令后：

1. 在目标函数的 poll 入口安装 PollEntryBP，并将其加入 `_ACTIVE_ROOTS` 集合。
2. 在 poll 入口内部安装 CallSiteBP（用于追踪 call edge）。
3. 在函数返回点安装 PopOnReturnBP（用于触发影子栈弹出）。
4. 当程序执行到该 poll 入口时，脚本读取 `__awaitee` 字段，发现子 Future，自动为子 Future 重复步骤 1–3。

这样，追踪范围被精确限定在从 trace root 出发、沿 await edge 和 call edge 实际执行到的路径上。未被命中的分支不产生任何开销。

¹⁶trace root（追踪根节点）：用户指定的异步追踪起点。系统从该函数出发，通过 `__awaitee` 字段顺藤摸瓜地发现被等待的子 Future，而不关心依赖树中与当前执行路径无关的函数。

```
1 def cmd_ardb_trace(symbol):
2     """安装 trace root 的跟踪断点链"""
3     poll_fn = find_poll_function(symbol)
4     if poll_fn is None:
5         raise ValueError(f"Cannot find poll for {symbol}")
6
7     # 1. 安装 poll 入口断点
8     bp = install_breakpoint(poll_fn, type='PollEntry')
9     _ACTIVE_ROOTS.add(symbol)
10
11     # 2. 在 poll 函数内部安装 call-site 断点
12     #     (用于追踪该 poll 函数内调用了哪些同步函数)
13     install_callsite_breakpoints(poll_fn)
14
15     # 3. 在 poll 函数返回点安装断点
16     install_return_breakpoint(poll_fn, type='PopOnReturn')
17
18     # 4. 预读 __awaitee 字段类型, 为子 Future 安装追踪
19     #     但实际生效要等到 poll 执行到该 .await 点时
20     return f"[ARD] trace root set: {symbol}"
```

代码段 3: ardb-trace 命令: 设置追踪根节点

未命中的分支不产生任何开销。

调试器还提供了自动推断 trace root 的能力: ardb-infer-trace-root 命令在调试器暂停时, 从当前物理调用栈 (GDB 的 backtrace) 中向外层逐帧回溯, 找到最外层的用户 crate 异步函数作为追踪根节点。这个功能目前还未集成进 Async Inspector 的图形界面, 但已通过命令行可用, 降低了用户手动指定根节点的门槛。

整个过程中, 系统只在程序实际执行到的执行路径上按需安装断点。未被当前执行路径触达的异步函数永远不会被插桩。这一改进省去了调试前的 DWARF 全量解析步骤, 将追踪开销从「与程序规模成正比」降低为「与当前执行路径长度成正比」。

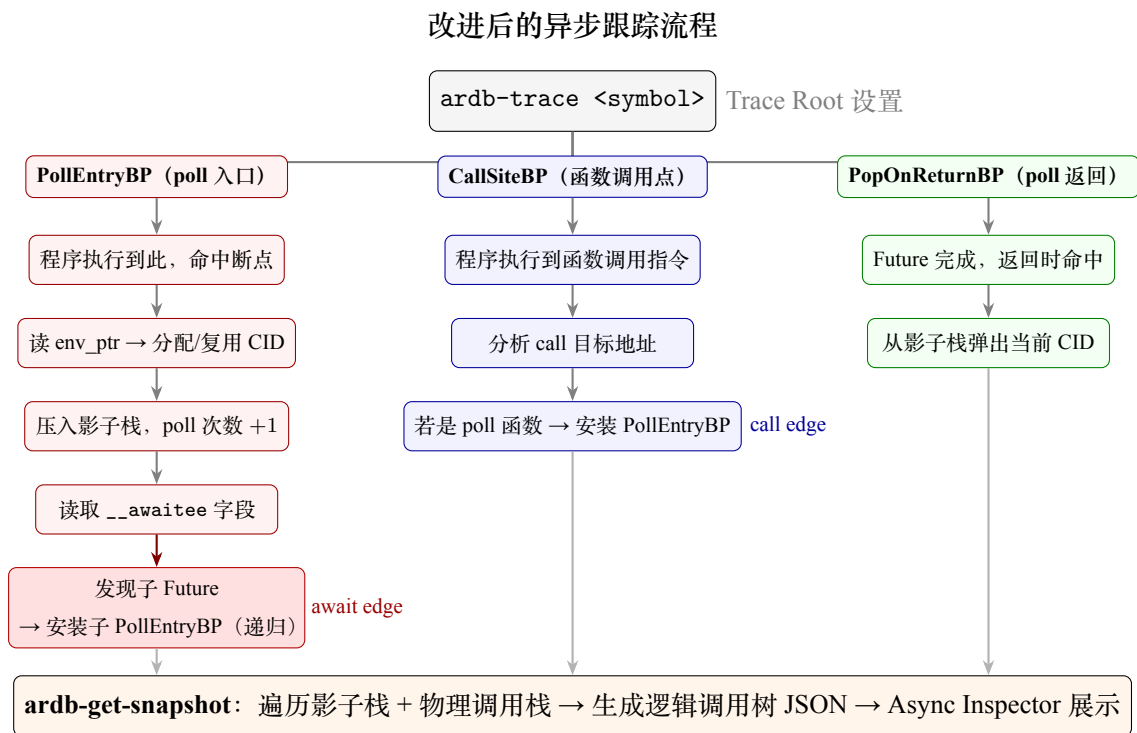


图 4: 调试器内部异步跟踪流程：从 trace root 设置到逻辑调用树快照生成的完整机制

3.2 跟踪状态跨特权级保持问题：异步跟踪与 OS 调试的融合

3.2.1 问题的具体表现

跨特权级调试和异步函数跟踪各自独立时都能正常工作，但一旦将它们放在同一个调试会话中，就会出现冲突。核心冲突在于：特权级切换会清除异步跟踪数据，本质上是两个功能的相关数据生命周期存在冲突——跨特权级调试要求在符号表切换时清理一组数据，但异步跟踪恰恰依赖这组数据来维持协程追踪的连续性。

具体来说，受冲突影响的数据分为四类：

数据类别	具体数据	冲突原因
断点数据 (6 类)	PollEntryBP、CallSiteBP、PopOnReturnBP、 边界断点、Hook 断点、用户断点	切换断点组时被 GDB 层 物理删除，不可恢复
协程状态 (5 项)	_CO_BY_KEY (实例映射)、 _CO_META (CID 元数据)、 _CO_POLL_SEQ (poll 序列)、 _TLS_STACK (影子栈)、 _CO_NEXT_ID (CID 计数器)	new_objfile 事件触发 _cleanup_run_scoped 全部清空
追踪记录 (3 项)	_ACTIVE_ROOTS (trace root 集合)、 _CALLSITE_INSTALLED_FOR_FN (去重集合)、 _CREATED_BPS (已创建断点引用)	同上， 与协程状态同时被清空
白名单数据 (5 项)	_WHITELIST_EXACT (精确匹配集)、 _WHITELIST_PREFIX (前缀匹配集)、 _WHITELIST_ADDR_MAP (函数 → 地址映射)、 _WHITELIST_ADDR_READY (映射就绪标记)、 _ASYNC_SYMBOL_SET (可用 poll 函数集)	前两项直接清空；地址映射因 目标地址空间变化而全部失效； _ASYNC_SYMBOL_SET 因 符号文件更换需重建

这四类数据中，断点数据是问题的表象——GDB 物理删除导致追踪中断；而协程状态、追踪记录和白名单数据才是问题的本质——它们记录了「当前追踪到了哪里」「每个协程实例是谁」「等待关系是怎样串联的」，一旦丢失，即使重新插入了相同的断点，之前的追踪上下文也已不可恢复。这正是为什么单纯在切换后重新设置断点无法解决问题：协程的 CID 分配、poll 次数累计、影子栈结构、白名单地址映射等运行期积累的状态，必须完整保存并恢复。

3.2.2 解决方案：跨生命周期的保存与恢复机制

解决冲突的思路是：在特权级切换清空数据之前，将四类数据完整序列化保存；切换完成后，按正确的依赖顺序逐步恢复。

保存阶段（在清除旧断点组之前执行，对应代码中 `arldb-save-trace-state` 命令）：

1. 保存断点元数据：遍历 `_CREATED_BPS` 中的所有 `PollEntryBP` 实例，记录每个断点的位置 (location)、poll 符号名 (poll_sym) 和内部标记 (internal)，供恢复时重新

安装。

2. 保存协程状态：序列化 `_CO_BY_KEY` (`(poll_sym, env_ptr) → CID` 的实例映射)、`_CO_META` (每个 CID 对应的 poll 符号和地址)、`_CO_POLL_SEQ` (每个 CID 的累计 poll 次数)、`_TLS_STACK` (每个线程的影子栈) 和 `_CO_NEXT_ID` (下一个可用 CID 编号)。
3. 保存追踪记录：序列化 `_ACTIVE_ROOTS` (当前正在追踪的 trace root 函数集合) 和 `_CALLSITE_INSTALLED_FOR_FN` (已安装 CallSiteBP 的函数集合)。
4. 保存白名单配置：序列化 `_WHITELIST_EXACT`、`_WHITELIST_PREFIX`、`_WHITELIST_ADDR_MAP` 和 `_ASYNC_SYMBOL_SET`。白名单的地址映射在特权级切换后将全部失效 (目标地址空间已变化)，必须在恢复时标记为待重建。

保存后的数据以断点组标签 (如 "kernel"、"user") 为键，存入 `_SAVED_STATES` 字典中。

恢复阶段 (在新符号表加载完成、新断点组激活后执行，对应代码中 `ardb-restore-trace-state` 命令，必须按以下顺序)：

```
1 // 恢复有严格的顺序要求，后续步骤依赖前一步的结果
2 async function restoreAsyncState() {
3     // 1. 恢复白名单配置，标记地址映射待重建
4     //     (特权级切换导致目标地址空间变化，所有函数地址映射失效)
5     await restoreWhitelistConfig();
6     markAddressMappingStale();
7
8     // 2. 恢复追踪记录
9     //     (ACTIVE_ROOTS 和 CALLSITE_INSTALLED_FOR_FN，不设置断点)
10    await restoreActiveRoots();
11    await restoreCallSiteInstalled();
12
13    // 3. 恢复协程状态数据
14    //     (CID 命名空间、poll 序列、影子栈——纯内存数据)
15    await restoreCoroutineState();
16
```

```
17 // 4. 清除残余旧断点，避免跨组切换累积重复断点
18 await cleanupOrphanBreakpoints();
19
20 // 5. 遍历保存的 poll_entries，逐个重建 PollEntryBP
21 //     (依赖步骤 1 的地址映射和步骤 3 的 CID 命名空间)
22 for (const entry of savedPollEntries) {
23     await rebuildPollEntryBreakpoint(entry);
24 }
25 }
```

代码段 4: 恢复异步跟踪状态的五步顺序

恢复顺序的设计依据是：(1) 白名单和地址映射是定位函数的基础——没有它，后续任何断点都无法正确设置；(2) 追踪记录决定了哪些 poll 入口需要恢复；(3) 协程状态必须在断点重建之前恢复——PollEntryBP 安装后可能立即触发，此时 CID 命名空间必须已经就绪，否则新 poll 事件会被误认为首次出现；(4) 清除残余旧断点防止跨组切换中 GDB 断点累积（包括 RISC-V GDB 15 中符号文件卸载后残留的孤儿 PollEntryBP）；(5) PollEntryBP 的实际安装必须在最后——它依赖前四个步骤的全部结果。

融合后的完整切换流程(对应 breakpointGroups.ts 的 updateCurrentBreakpointGroup 方法，约 160 行)：

1. 状态机检测到特权级切换条件(如 PC 进入内核空间且栈帧函数名匹配 user→kernel 边界断点)。
2. 保存异步状态：通过 MI2 协议执行 ardb-save-trace-state，将四类数据序列化保存。
3. 清除旧断点组：GDB 层删除所有断点；按编号或函数名删除旧组的函数名边界断点和 Hook 断点；卸载旧符号文件。
4. 激活新断点组：加载新符号文件；恢复普通断点和 OS 特殊断点（边界断点、Hook 断点）；发送 BreakpointEvent 通知 VS Code 界面更新。
5. 恢复异步状态：通过 MI2 协议执行 ardb-restore-trace-state，按五步顺序恢复白名单 → 追踪记录 → 协程状态 → 清除残余 → 重建 PollEntryBP。

6. 继续执行。

通过这套跨生命周期的保存与恢复机制，四类数据在特权级切换前后保持连续——协程 ID 不丢失、poll 次数累计不间断、await edge 可以跨地址空间串联、白名单地址映射在新地址空间中正确重建。调试器从「两种功能互相破坏」变为「两个子系统协同工作」，这是实现异步操作系统调试的必要条件。

3.3 调试器的可视化问题：图形化界面设计

3.3.1 问题的具体表现

2025 年的异步跟踪工具只是一个 GDB Python 命令行脚本——开发者需要在终端中手动输入命令、查看文本输出来理解异步执行流。对于包含数十个协程的实际程序，文本输出中十几层缩进的 `awa` 和 `call` 行几乎无法阅读。开发者无法快速回答「哪些协程在等待同一个依赖」「当前被 poll 次数最多的协程是哪个」「完整的调用树有多大」这类问题。

更根本的是，VS Code 的调试界面只认识三样东西：线程、栈帧和变量。这对同步程序的调试是完备的——每个线程的调用栈天然就是一棵树，栈帧的父子关系就是真实的控制流。但异步程序有三类关键信息是 VS Code 原生界面无法承载的：

1. 等待拓扑：谁在 `await` 谁、完整的依赖树长什么样。VS Code 的调用栈只能看到「谁调用了当前 poll」，看不到「当前函数在等谁」。await edge 反映的是源码语义层的等待关系，与物理控制流不在同一个维度——不能伪装成栈帧（破坏单步调试语义），也不能伪装成虚拟线程（导致线程列表臃肿且含义混乱）。
2. 协程元数据：每个协程的 CID（跨 poll 周期追踪同一实例的唯一标识）、被 poll 次数（该实例自创建以来的累计被驱动次数）、运行状态（正在运行 / 等待中 / 已完成）。这些数据是理解异步执行进度的关键——比如一个被 poll 了 50 次仍处于「等待中」的协程很可能存在性能问题——但 VS Code 的变量面板无法自动提取和呈现这类跨 poll 周期累积的元数据。
3. 节点身份：一个函数是 `async` 协程还是 `sync` 同步函数，在 VS Code 的调用栈中无法直观区分。两者在调试行为上有本质差异——`async` 函数跨越多个 poll 周期、有自己的 CID 和状态生命周期，而 `sync` 函数只在本次调用内有效。

因此，需要自建一个专门的图形视图来承载这三类异步语义——这就是 Async Inspector 面板的设计出发点。

3.3.2 三层架构

解决方案是在 VS Code 插件端自建一个专门承载异步语义的图形化面板——Async Inspector。面板采用三层架构：

1. 数据层：GDB Python 脚本在每次调试器暂停时，扫描影子栈和追踪数据，通过 CLI 命令 `ardb-get-snapshot` 输出当前异步 + 同步调用栈的完整快照（结构化的 JSON 数据）。调试适配器通过 MI2 协议层执行该命令并解析返回的 JSON。
2. 传输层：调试适配器 (Debug Adapter¹⁷) 将解析后的快照数据通过 VS Code Webview 消息通道发送给面板。面板通过 `customRequest` 向调试适配器发起白名单生成、Trace 启动等操作。
3. 展示层：Webview¹⁸ 面板中的前端代码解析快照数据，构建树形数据结构，渲染为可交互的树形图。

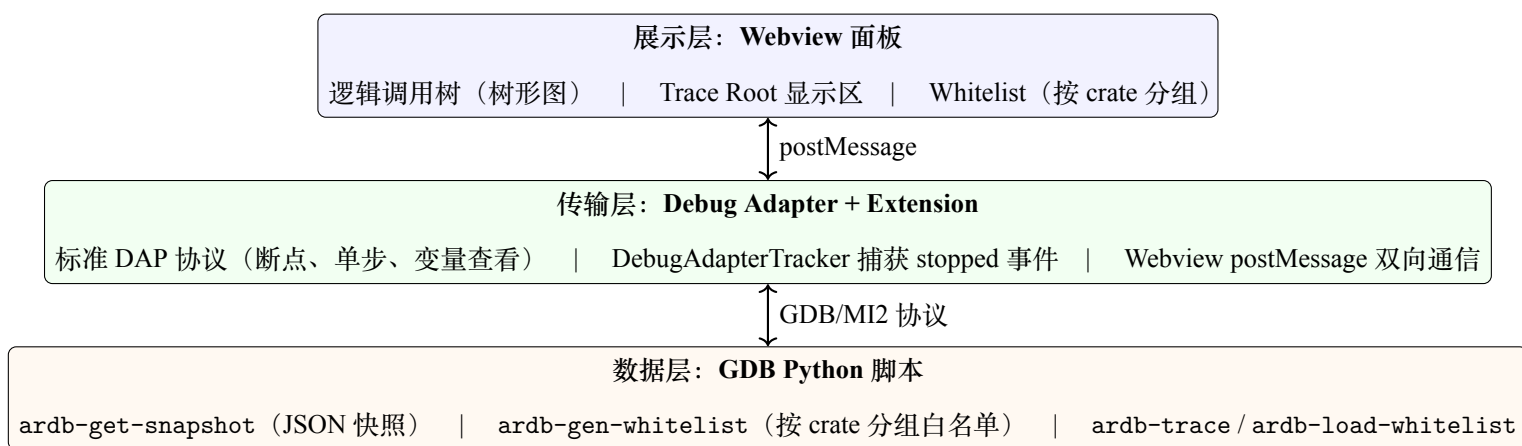


图 5: Async Inspector 三层架构

¹⁷Debug Adapter（调试适配器）：VS Code 调试架构的核心组件。它是一个独立的进程，负责将 VS Code 的标准调试协议（DAP）翻译为具体调试器后端（如 GDB）的命令。通过实现 Debug Adapter，自定义调试器可以无缝集成到 VS Code 的调试界面中。

¹⁸Webview：VS Code 内嵌的网页视图，可以用 HTML、CSS、JavaScript 构建自定义用户界面，与编辑器原生面板并存。

3.3.3 界面布局与 workflow

面板的左侧是逻辑调用树，以多根节点树形图展示从 trace root 到当前暂停点的完整调用路径。async 协程节点使用红色，sync 同步函数节点使用绿色，开发者可以一目了然地识别哪些路径经过异步边界。每个异步节点标注三项信息：协程 ID（跨快照追踪同一 Future 实例）、被 poll 次数（该实例自创建以来的累计 poll 次数）、运行状态（正在运行 / 等待中 / 已完成）。点击任意节点，编辑器自动跳转到对应源码行。

面板的右侧上方显示当前已设置的追踪根函数列表，下方是按 crate¹⁹分组的白名单。用户 crate 和框架 crate（第三方库）分区显示，每个 crate 可展开查看内部函数列表，每个函数提供 Trace 和 Locate 按钮。

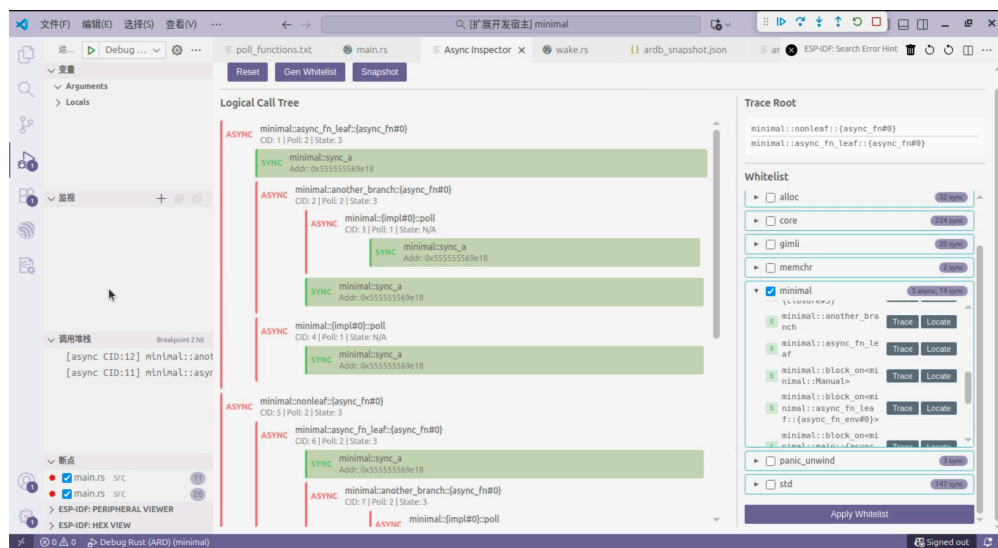


图 6: Async Inspector Webview 面板：左侧为逻辑调用树视图（红色 async 节点、绿色 sync 节点，标注 CID 和 poll 次数），右侧为按 crate 分组的白名单列表（含 Gen/Apply/Trace 按钮）

完整的工作流如下：开发者从启动调试到查看异步执行流，共六步操作与系统响应交替进行。

¹⁹crate: Rust 语言的基本编译单元，类似 C 语言的「库」。一个项目通常由多个 crate 组成，包括用户自己编写的 crate 和通过包管理器引入的第三方依赖。

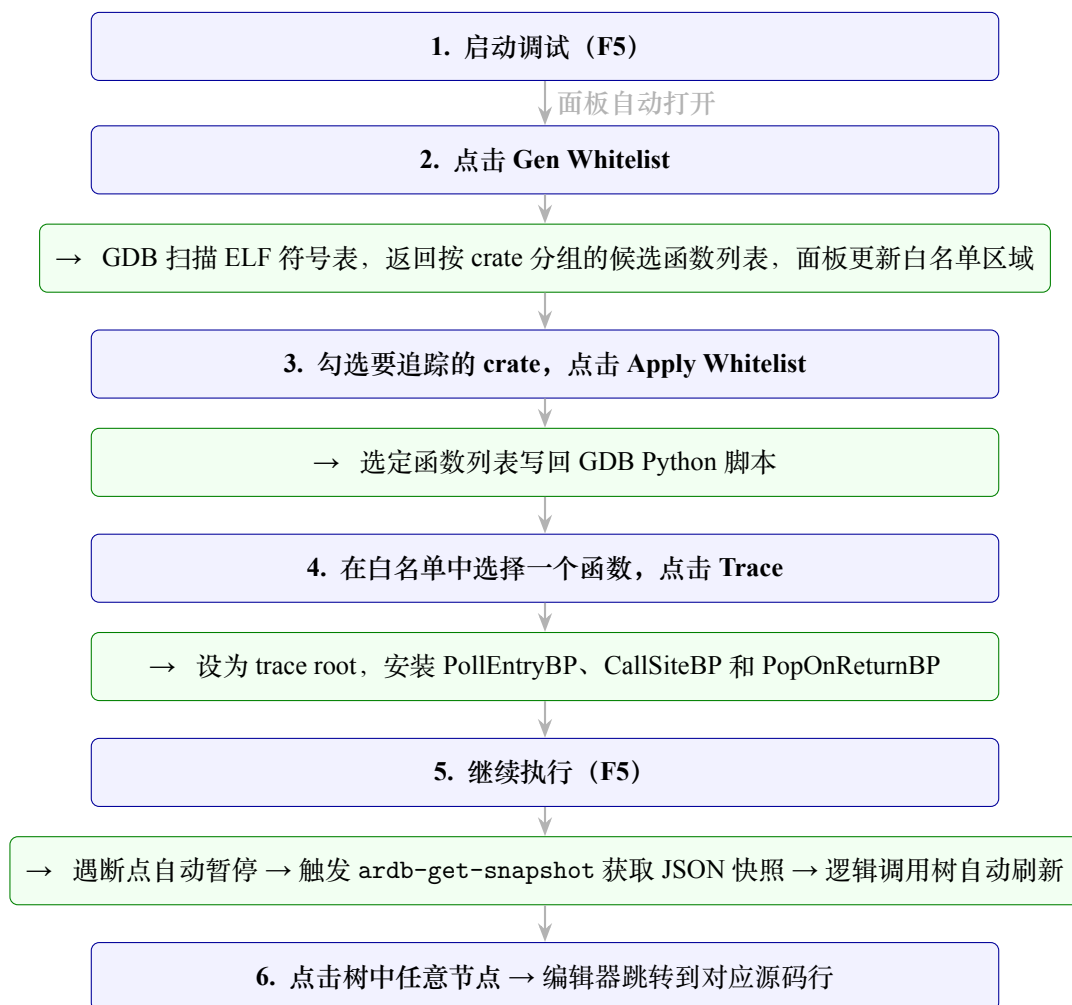


图 7: Async Inspector 完整 workflow：蓝色为用户操作，绿色为系统响应，六步完成从启动调试到查看异步执行流

3.3.4 物理流 + 逻辑流双轨协同

Async Inspector 不是替代 VS Code 原生的 Call Stack 面板，而是与之平行运行。原生 Call Stack 继续处理所有标准调试操作——断点管理、单步执行、变量查看。Async Inspector 仅在调试器暂停时自动刷新，呈现当前时刻的异步执行流快照。开发者的典型使用方式是：在 Call Stack 中查看「当前执行到了哪个栈帧」→ 切换到 Async Inspector 查看「这个协程在整个异步执行流中的位置以及它正在等待谁」。两者共同构成「物理流 + 逻辑流」的双轨调试体验。

3.4 调试器可移植性问题：面向不同操作系统的通用化改进

3.4.1 问题的具体表现

前序工作（2023–2024 年）实现的跨特权级调试器在教学操作系统（如 rCore）上适配良好，但这得益于教学 OS 的静态特征：内核与用户程序源码同处一个工作区、系统调用路径单一且可预期、用户程序在调试前已知。这些特征使得调试器可以通过相对固定的配置方案来工作。

然而，在 Rust 操作系统领域，组件化 OS（如基于 ArceOS 框架的 StarryOS²⁰）是重要且普遍的一类 OS。它们的设计高度灵活——内核功能被拆分为可替换的 crate、切换边界位于外部依赖中、用户程序在运行时动态加载——这些正是教学 OS 不具备的特征。前序调试方案在设计时依赖了教学 OS 的静态特征，导致其通用性局限于结构相似的 OS，无法覆盖组件化 OS 这类灵活度更高的目标。

具体来说，前序方案存在三个通用性缺陷：

1. 特殊断点设置依赖本地源码文件路径。边界断点和 Hook 断点都需要在调试前通过文件路径加行号写入配置文件（launch.json）。但组件化 OS（如 StarryOS²¹）的特权级切换函数位于外部 crate（如 axhal 中的 enter_user），其源码路径随 crate 版本号和开发环境变化，无法硬编码在配置中。此外，用户态的系统调用入口分散在 C 库²²的各个封装中，也不存在一个固定的文件路径可以配置。
2. 通过硬编码 Rust String 的内部字段路径来读取当前进程名。Hook 断点在 execve²³系统调用处触发时，需要读取传入的程序路径（Rust 的 String 类型）来确定即将启动的用户进程名。已有实现直接硬编码 String 内部字段的访问路径——但这条路径会随 rustc 版本变化。旧版 Rust 中数据指针位于 String.vec.buf.ptr.pointer，新版（1.73+）移至 String.vec.buf.inner.ptr.pointer。一旦目标 OS 使用的

²⁰StarryOS：基于 ArceOS 框架的组件化 Rust 操作系统。其内核通过 Cargo 包管理器引入外部依赖（如 axhal），用户程序是独立编译的 ELF 文件，通过交互式 shell 动态启动。

²¹StarryOS：基于 ArceOS 框架的组件化 Rust 操作系统。与教学 OS 不同，其内核通过 Cargo 包管理器引入外部依赖（如 axhal、axsync），用户程序是独立编译的 Linux ELF 文件，通过交互式 shell 动态启动。

²²C 库（libc）：C 语言标准库，提供了系统调用的封装函数（如 write、read、open 等），应用程序通过它间接使用系统调用。每个封装函数内部都可能内联一条 ecall 指令。

²³execve：Unix 系统中用于执行新程序的系统调用。操作系统通过它加载并运行一个新的可执行文件，调用时需传入新程序的路径。

Rust 版本与调试器硬编码的路径不匹配，进程名读取就失效，后续断点组切换链条全部断裂。

3. 所有用户进程组需要在调试开始前写入配置文件。教学 OS (rCore) 的用户程序是固定的，调试前即可在 launch.json 中配置好所有进程组。但更广泛的 OS 中，用户程序通过交互式 shell 动态启动——新的进程名在 execve 执行时才出现，调试器无法预知需要创建哪些进程组，也无法提前配置。

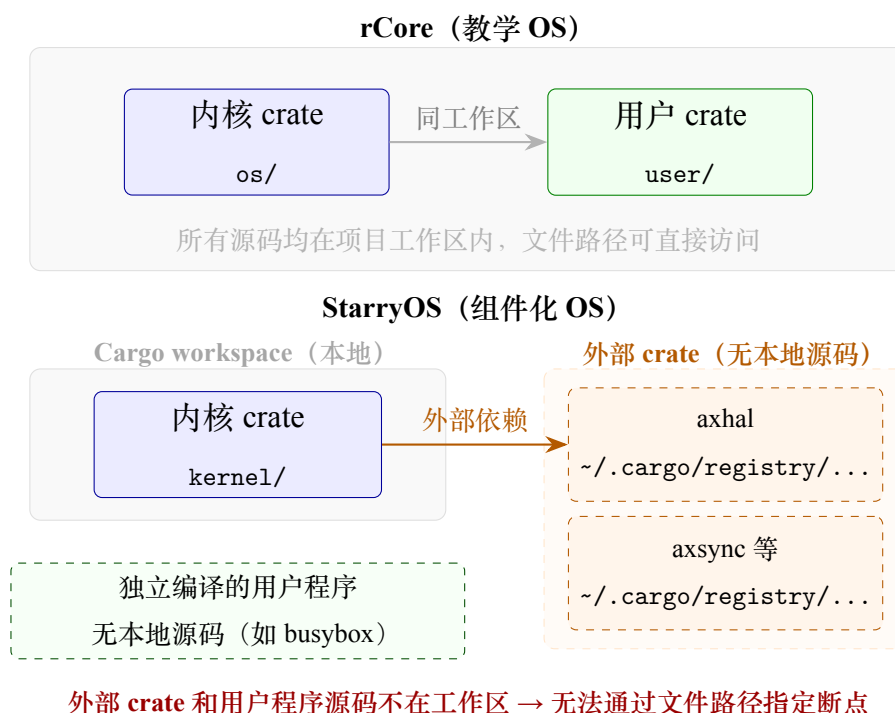


图 8: rCore 与 StarryOS 的架构差异。教学 OS 所有源码同处一个工作区，组件化 OS 的切换边界位于外部 crate，用户程序独立编译。

以下三项改进分别对应上述三个通用性缺陷，使调试器摆脱对教学 OS 静态特征的依赖，能够覆盖组件化 OS 等更广泛的目标。

3.4.2 改进一：用函数名指定特殊断点，摆脱对本地源码文件路径的依赖

边界断点²⁴和 Hook 断点都需要在调试前写入 launch.json，而前序工作只有一种指定方式——文件路径加行号。对于组件化 OS，切换函数位于外部 crate 中，其路径类

²⁴边界断点 (border breakpoint): 标记内核 ↔ 用户态切换位置的断点，由调试器的状态机自动管理。命中时不暂停给用户，而是触发符号表和断点组的切换流程。

似 `~/ .cargo/registry/src/.../axhal-0.3.0/context.rs`，随 crate 版本和开发环境变化，无法硬编码。

改进方案是在特殊断点的配置中增加函数名指定方式：`{ "function": "enter_user" }`。GDB 本身具备通过符号表查找函数地址的能力——`break < 函数名 >` 不关心源码文件在哪，只要目标函数被链接到了 ELF 中就能定位。

```
1 border_breakpoints?: Array<
2   | { filepath: string; line: number; direction?: ... } // 文件路径方式（原有）
3   | { marker: string; direction?: ... } // 源码标记扫描方式（原有）
4   | { function: string; direction?: ... } // 函数名方式（新增）
5 >;
```

代码段 5: 特殊断点配置的三种方式：文件路径、源码标记、函数名

函数名断点还带来了一个额外收益：解决了用户态系统调用入口分散的问题。组件化 OS 的用户程序是独立编译的 Linux ELF 文件，通过 C 库发起系统调用——`write()`、`read()`、`open()` 等上百个封装函数各自内联 `ecall`²⁵ 指令，用户态不存在统一的入口可以配置断点。但从内核态看，无论用户程序通过哪个封装函数发起系统调用，CPU 陷入内核后都必须经过同一个系统调用分发函数（如 StarryOS 的 `handle_syscall`、rCore 的 `trap_handler`、Linux 的 `do_syscall_64`）。只需配置一个函数名断点 `{ "function": "handle_syscall" }`，即可覆盖所有用户程序的全部系统调用返回路径。

而在引入函数名断点后，它也带来了一个额外问题：`kernel→user` 方向（如 `enter_user`）和 `user→kernel` 方向（如 `handle_syscall`）的边界断点同时落在了内核地址空间中。前序状态机通过 PC 寄存器地址范围隐式判断执行方向的前提不再成立。解决方案是为每个边界断点引入方向属性（`direction`），取值为 `kernel_to_user` 或 `user_to_kernel`，状态机匹配时同时校验方向和当前状态。

```
1 private borderMatches(border: Border, filepath: string,
2   lineNumber: number, funcName?: string,
3   direction?: 'kernel_to_user' | 'user_to_kernel'): boolean {
4   // 第一层：方向不匹配 → 直接排除
```

²⁵`ecall`: RISC-V 架构的指令，用户态程序通过它陷入内核态请求系统服务（如读写文件、创建进程等）。

```

5     if (direction != undefined && border.direction != direction) return false
6     ;
7     // 第二层：函数名断点 → 比较函数名
8     if (border.func != undefined)
9         return funcName != undefined && funcName === border.func;
10    // 第三层：文件路径断点 → 比较 filepath + line
11    return filepath === border.filepath && lineNumber === border.line;
12 }

```

代码段 6: 边界断点匹配：函数名 + 方向双重校验

函数名断点还面临一个工程细节：跨断点组切换时，没有文件路径可依赖，必须在 GDB 层面执行「删除旧断点 → 插入新断点」的完整流程。而 GDB 的断点编号是异步分配的——发出 `break` 命令后，编号通过 MI 异步通知返回，存在一个短暂的时间窗口内编号尚未就绪。解决方案是双路径删除：优先按编号删除（精确且高效），编号未就绪时回退到按函数名删除。

3.4.3 改进二：放弃硬编码字段路径，改为依赖 GDB 提供的稳定接口读取变量

Hook 断点在内核的 `execve` 系统调用处触发时，需要读取传入的程序路径（Rust 的 `String` 类型）来确定即将启动的用户进程名，进而选择对应的断点组。这一操作的困难在于：`String` 的内部布局随编译器版本变化，硬编码字段路径只能兼容特定 Rust 版本。

改进的方向是放弃直接访问 Rust 内部字段，改为依赖 GDB 提供的稳定接口。具体方案分三步：

第一步：通过 GDB 的 **pretty printer**²⁶ 获取格式化的变量输出。执行 `p < 变量名 >` 后，GDB 调用 Rust pretty printer，输出统一的格式化文本（含 `pointer` 和 `len` 字段），不依赖任何 Rust 版本特定的字段路径。

第二步：在 **MI2** 协议层自建控制台输出捕获机制。pretty printer 的输出通过 GDB/MI 协议的控制台输出流²⁷发送——这种输出不携带命令编号²⁸，无法直接关联到发起命令。

²⁶pretty printer: GDB 的扩展机制，用于将复杂数据结构的内部字段按可读格式展开输出。Rust 编译器为 GDB 提供了 Rust 标准库类型的 pretty printer，能够自动理解 `String` 等类型的内部布局。

²⁷控制台输出流（console stream）：GDB/MI 协议中以 `~"...."` 开头异步输出的文本，是 GDB 命令行输出的通道。

²⁸命令编号（token）：GDB/MI 协议中每条命令携带的唯一数字标识，用于将命令与其响应配对。控制

协议也没有提供「获取上一条 GDB 命令行输出」的编程接口。但 GDB/MI 协议有一个关键的隐式保证：GDB 严格按序处理命令，某条命令的所有控制台输出必定在其结果记录²⁹之前到达。基于这一观察，在协议层实现 `captureConsoleOutput()`：发送命令后注册临时监听器捕获所有 console 输出，直到对应命令的 result record 到达后移除监听器并返回收集到的全部行。

```

1 captureConsoleOutput(command: string): Promise<string> {
2     const lines: string[] = [];
3     const msgHandler = (type: string, msg: string) => {
4         if (type === "console") lines.push(msg);
5     };
6     this.on("msg", msgHandler);           // 注册临时监听器
7     return this.sendCommand(
8         `interpreter-exec console "${command.replace(/[\\"']/g, "\\$&")}"`
9     )
10    .then(() => {                          // result record 到达 = 所有输出已就
11        this.removeListener("msg", msgHandler);
12        return lines.join("");
13    })
14    .catch((e) => {
15        this.removeListener("msg", msgHandler); // 错误时也清理，避免泄漏
16        throw e;
17    });
18 }

```

代码段 7: 利用协议顺序性保证实现控制台输出捕获

第三步：从 pretty printer 输出中提取地址和长度，直接读取目标内存。

```

1 const output = await miDebugger.captureConsoleOutput(`p ${variableName}`);
2 // 从 pretty printer 输出中正则提取 pointer 和 len
3 // 输出示例: ... pointer = 0xffffffffc08028d, len = 6 ...

```

台输出流不携带 token，因此无法直接判断某行输出属于哪条命令。

²⁹结果记录 (result record)：GDB/MI 协议中每条命令执行完毕后的结构化响应，以 `^done` 或 `^error` 开头，标志着该命令的处理结束。

```

4 const ptrMatch = output.match(/pointer\s*=\s*(0x[0-9a-fA-F]+)/);
5 const lenMatch = output.match(/len\s*=\s*(\d+)/);
6 if (ptrMatch && lenMatch) {
7     const ptr = ptrMatch[1];
8     const len = parseInt(lenMatch[1]);
9     // 通过 GDB 直接从内存读取原始字节
10    const bytes = await miDebugger.sendCommand(
11        `data-read-memory-bytes ${ptr} ${len}`);
12    return decodeMemoryBytes(bytes); // 解码为 UTF-8 字符串
13 }

```

代码段 8: 通过 console 输出捕获 + 内存直读获取 String 内容

整个过程中，调试器不依赖任何 Rust 版本特定的字段路径——数据结构的理解由 GDB pretty printer 负责（随 Rust 工具链同步更新），调试器只做自己擅长的事：内存读取和字节重构。

3.4.4 改进三：进程组管理从静态预配置到运行时动态自动注入

前序工作的调试模型假设所有用户进程在调试前已知，进程组和边界断点都在配置阶段创建完毕。但更广泛的 OS（如 StarryOS）通过交互式 shell 运行——用户在终端中敲入命令启动任意程序，新的进程名在 `execve` 执行时才出现，对应的断点组也只能在运行时动态创建。

在已有实现中，动态创建的进程组是一个没有边界断点（`borders = []`）的空壳，缺少 `user`→`kernel` 方向的边界断点。当用户程序发起系统调用时，CPU 执行 `ecall` 陷入内核，但调试器无法检测到这次 `user`→`kernel` 切换——状态机停留在 `user` 态，切换链条断裂。

解决方案是维护一个待注入队列，保证一条不变式：无论进程组是何时、以何种方式创建的，每个用户进程组都必须拥有 `user`→`kernel` 方向的边界断点。所有 `user`→`kernel` 方向的函数名边界断点在配置阶段被推入 `pendingUserToKernelFuncBorders` 队列。此后，无论进程组是何时、通过何种方式创建的（Hook 断点触发、用户首次设置断点时触发），都自动从队列中继承这些边界断点：

```

1 // 字段声明
2 private pendingUserToKernelFuncBorders: Border[] = [];
3

```

```

4 // 动态创建新进程组时自动注入 (updateCurrentBreakpointGroup)
5 public updateCurrentBreakpointGroup(updateTo: string, ...) {
6     let newIndex = this.groups.findIndex(g => g.name === updateTo);
7     if (newIndex === -1) {
8         // 新组不存在：创建时自动注入待定 user→kernel 边界断点
9         const initialBorders = this.pendingUserToKernelFuncBorders.map(b => b);
10        this.groups.push(new BreakpointGroup(
11            updateTo, [], new HookBreakpoints([]), initialBorders));
12        newIndex = this.groups.length - 1;
13    }
14    // ... 切换断点组
15 }
16
17 // 用户首次在新进程源码中设置断点时也会触发 (saveBreakpointsToBreakpointGroup)
18 public saveBreakpointsToBreakpointGroup(
19     args: DebugProtocol.SetBreakpointsArguments, groupName: string) {
20     let found = this.groups.findIndex(g => g.name === groupName);
21     if (found === -1) {
22         const initialBorders = this.pendingUserToKernelFuncBorders.map(b => b);
23         this.groups.push(new BreakpointGroup(
24             groupName, [], new HookBreakpoints([]), initialBorders));
25         found = this.groups.length - 1;
26     }
27     // ... 保存断点
28 }

```

代码段 9: 待注入队列与动态进程组自动注入

新进程名的来源是改进二中描述的跨版本变量读取机制：Hook 断点在 `execve` 处命中，通过 GDB pretty printer + console 捕获 + 内存直读获取程序路径（如 `/test_user`），该路径直接作为新进程组名，触发上述动态创建和边界断点自动注入。

三项改进合在一起，调试器的通用性从「适用于静态教学 OS」扩展到「覆盖灵活度更高的组件化 OS」。调试器不再依赖源码同工作区、单一系统调用入口、进程提前已知等教学 OS 特有的静态特征，从而能够适配以 StarryOS 为代表的组件化 OS，以及更

广泛的 Rust 操作系统。

3.5 调试器的硬件适配问题：VisionFive 2 真机调试与线上化扩展

3.5.1 问题的具体表现

前述调试机制主要解决不同地址空间之间的符号表与断点管理问题，但当调试对象由 QEMU 虚拟机迁移到真实 RISC-V 开发板时，还需要处理一组新的工程约束。虚拟机能够直接向 GDB 暴露处理器状态，真实硬件则必须经过 JTAG 调试接口；处理器暂停后，运行在同一芯片上的网络和 SSH 服务也会同步停止；串口、JTAG 与板载系统分别承担启动观察、处理器控制和文件访问任务，任何一条链路不稳定都会影响完整调试流程。此外，真实处理器对调试 CSR 的访问权限和单步期间的中断行为也与 QEMU 存在差异，原有单步策略不能未经适配直接使用。

基于上述问题，本项目在已有工作的基础上，对 VisionFive 2 的硬件调试功能进行了复现和适配。一方面建立 PC、J-Link、OpenOCD、GDB 与开发板之间的真机控制链路，另一方面保留串口作为独立的启动输出通道，并进一步将真实开发板纳入线上平台的预约和调度体系。

3.5.2 VisionFive 2 真机调试链路

真实硬件调试链路如图 9 所示。GDB 负责加载目标操作系统的符号信息并发送断点、单步和寄存器读取命令；OpenOCD 将 GDB 远程协议转换为面向目标芯片的调试操作，并在本机 3333 端口提供 GDB Server；J-Link 通过 JTAG 引脚把调试命令送入 VisionFive 2。USB-TTL 串口不参与处理器控制，而是用于观察 U-Boot 和目标操作系统的输出，使调试者能够同时获得“处理器状态”和“系统运行日志”两类信息。

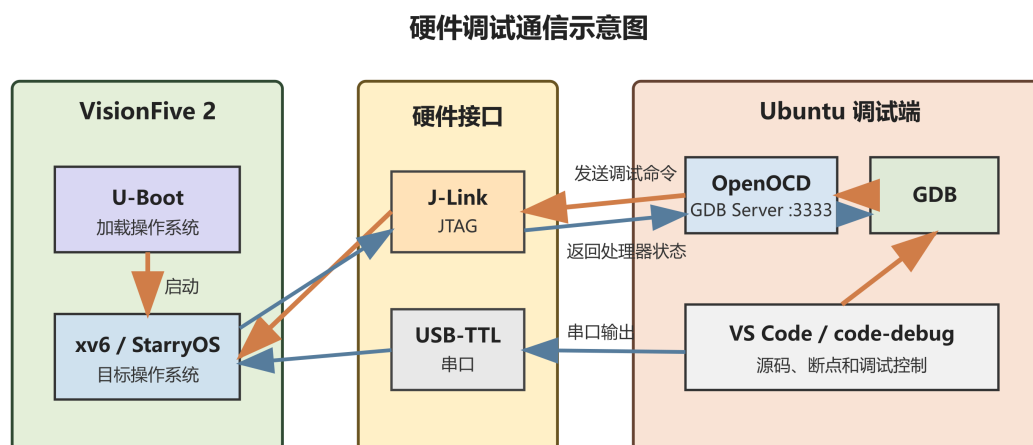
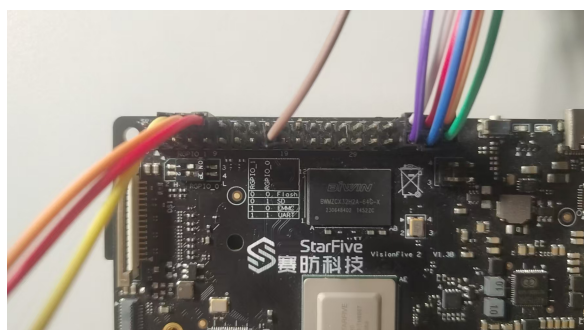


图 9: VisionFive 2 真机硬件调试通信链路

调试环境采用 J-Link V12 与 OpenOCD 0.12.0。JTAG 扫描能够识别 VisionFive 2 的 E24 和 U74 调试模块，TAP ID 为 0x07110cfd，并检测到 5 个 XLEN=64 的 hart。GDB 连接 OpenOCD 后可以查看各 hart 的线程状态，读取 PC、SP 和 RA 等寄存器，并控制处理器暂停与恢复。由于调试请求会暂停处理器，开发板的网络和 SSH 服务在暂停期间暂时不可用；恢复处理器后，两项服务能够继续运行。这一现象既验证了调试命令已经作用于真实处理器，也说明调试会话结束时必须显式恢复开发板运行状态。

硬件连接由 JTAG 与串口两部分组成。J-Link 以约 3.3 V 的目标参考电压工作，并连接 TMS、TRST、TCK、TDI、TDO 和 GND；USB-TTL 模块采用 115200 bit/s、8 个数据位、1 个停止位、无流控的串口参数，只连接 GND、RXD 和 TXD，不向开发板提供 VCC。实际串口连接及完整调试环境如图 10 所示。



(a) USB-TTL 串口连接



(b) J-Link、串口与开发板完整连接

图 10: VisionFive 2 硬件调试环境

3.5.3 内核态、用户态单步及跨特权级调试适配

在真实硬件上单步调试操作系统时，普通指令与特权级切换指令的行为并不相同。RISC-V 调试控制寄存器 dcsr 中的 stepie 字段决定单步期间是否允许中断。VisionFive 2 上该字段为 0，处理器在单步状态下关闭中断，因此对会引起异常或中断的指令直接执行单步，可能无法进入预期的处理流程。同时，开发板调试模块不支持通过 OpenOCD 的 set_reg 命令改写相关 CSR，不能简单地把 stepie 置 1 来绕过这一限制。

针对这一问题，调试器将边界断点与断点组机制结合起来。内核态和用户态分别维护独立的符号表与断点集合；用户程序执行 ecall 时，调试器在页表发生变化之前保存用户态断点，并在进入内核地址空间后启用内核态断点组；内核通过 sret 返回用户态时，再执行相反的切换。由于异常入口使用的跳板页在两种地址空间中具有一致的虚拟地址，它可以在页表切换过程中提供稳定的状态转换边界。

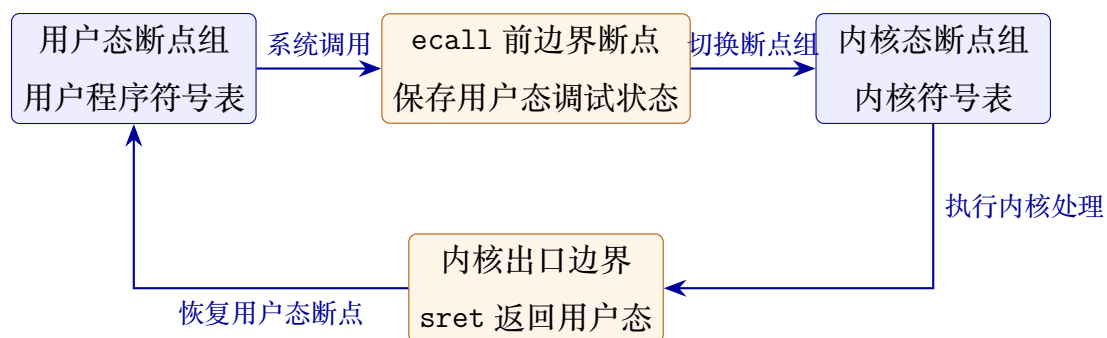


图 11: 内核态与用户态之间的断点组切换过程

中断入口位于 trampoline.S 汇编文件中，使用内核符号表时，GDB 难以像 C 语言源文件一样稳定地完成源码定位。为避免在页表切换后丢失调试状态，边界断点设置在 ecall 指令的前一条指令处，使调试器能够在进入异常处理流程前完成状态保存。该方法不依赖修改硬件 CSR，在保留断点组的前提下完成内核态、用户态单步调试及两个特权级之间的双向切换，构成了 code-debug 面向真实 RISC-V 开发板的关键适配。

3.5.4 线上硬件交互平台实现

真机调试解决的是“如何控制处理器”，但用于教学实验时还需要进一步回答“谁在什么时间使用哪块开发板”。为此，本项目设计并实现了面向 VisionFive 2 的线上硬件交互实验平台，整体结构如图 12 所示。平台以 Moodle 3.9.2 作为实验教学入口，Flask 提供用户注册、实验预约和设备查询服务，MariaDB 保存用户、预约、设备及调度状态，Docker

为不同用户提供相互隔离的实验环境。SSH、TFTP 和 RISC-V 工具链分别承担开发板访问、文件传输与程序编译任务。

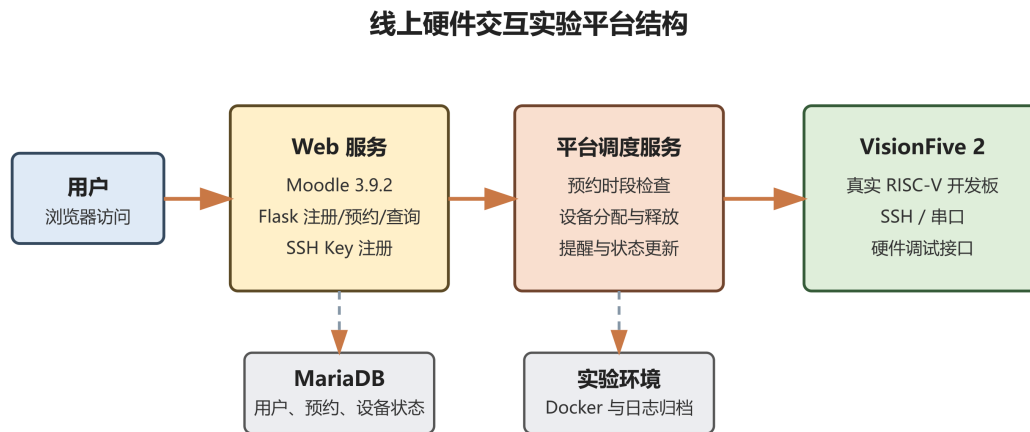


图 12: 线上硬件交互实验平台结构

用户注册 SSH 公钥后，可以通过网页预约实验时段并查询设备分配结果。调度服务在预约开始时将真实 VisionFive 2 分配给当前用户，在结束时释放开发板并恢复设备状态，同时完成提前提醒、状态更新、终端日志与实验会话归档。真实开发板已经作为独立设备接入平台，查询页面能够显示预约时段、SSH 连接信息和 RISC-V 架构，如图 13 所示。



图 13: 真实 VisionFive 2 预约与连接信息查询页面

3.5.5 线上平台与硬件调试能力结合

目前,真实 VisionFive 2 的预约、分配和释放流程已经在平台中实现,J-Link、OpenOCD、GDB 与串口组成的硬件调试链路也已独立建立。两部分的结合方式是在预约生效后,由平台同时授予开发板和调试资源的使用权,向用户提供当前会话的连接信息;预约结束后,平台停止调试会话、恢复处理器运行状态、归档串口日志并释放设备。这样可以把原本依赖现场接线和人工管理的调试过程转化为受预约时段约束的远程实验服务。

下一步将把 OpenOCD 进程启停、GDB 连接信息和串口日志进一步纳入调度流程,使用户在获得开发板权限后能够直接开展远程操作系统调试。在此基础上,继续尝试将现有跨特权级调试方法扩展到 StarryOS,并与 code-debug 的源码断点、单步和状态切换功能结合。

4 测试与验证

本章从基础的单元测试出发,验证状态机的逻辑正确性;再通过三个真实操作系统逐步验证调试器的各项能力:StarryOS 验证跨特权级调试的通用性,embassy 验证异步跟踪的运行时无关性,Async-os 验证统一调试平台的终极场景。

4.1 状态机单元测试与集成测试

在接入真实 QEMU 环境之前,先通过自动化测试验证状态机核心逻辑和完整调试流程的正确性。这部分测试不依赖 QEMU 虚拟机,可以快速检验调试器跨特权级调试能力。

4.1.1 OSStateMachine 单元测试 (37 个)

编写了 37 个 OSStateMachine 单元测试,覆盖四状态机的所有合法状态转换路径和边界条件:

- 正常转换路径:kernel $\rightarrow$ kernel_single_step_to_user $\rightarrow$ user $\rightarrow$ user_single_step_to_kernel $\rightarrow$ kernel,完整的特权级切换闭环。
- PC 快速路径:当状态机在 user 态、PC 已位于内核空间且栈帧函数名匹配 user $\rightarrow$ kernel 边界断点时,直接触发状态切换,跳过逐指令单步。

- 边界条件：同一地址空间内两个方向的边界断点（kernel→user 和 user→kernel 同在内核空间），验证方向属性能否正确区分切换方向。
- 异常路径：断点组未就绪、符号文件加载失败、GDB 断点编号异步返回超时等异常场景下的状态机行为。

4.1.2 MockMI2 集成测试（4 个场景）

testOSDebugFlow.ts（约 400 行）通过 MockMI2 模拟 GDB 后端，提供可控的寄存器值、栈帧信息和断点编号，在无需真实 QEMU 环境的情况下验证完整的 doAction + stateTransition 流程。四个集成测试场景：

- 内核启动 → 首次进入用户态。内核初始化完成后，状态机在 kernel→user 边界断点处停止（如 enter_user），进入单步模式，逐指令执行直到 PC 进入用户地址空间，切换断点组至目标用户进程组，继续执行。
- **Hook 断点动态进程发现**。execve 系统调用处 Hook 断点命中，执行行为函数读取新程序路径，创建新断点组并自动注入 user→kernel 边界断点，切换至新组。
- 用户态 → 内核态的 PC 快速路径。用户程序发起系统调用，PC 进入内核空间，状态机通过函数名匹配 user→kernel 边界断点（如 handle_syscall），走 PC 快速路径直接切换至内核断点组，无需进入单步模式。
- 多次特权级切换的完整性。连续三次完整的 kernel→user→kernel 切换循环，验证符号文件卸载/加载无遗漏、断点无残留无丢失、协程状态在保存/恢复后保持连续。

所有 37 个单元测试和 4 个集成测试场景均通过，为后续在真实 OS 上的验证提供了可靠的基础保证。

4.2 embassy 异步跟踪运行时无关性验证

embassy³⁰是本项目验证异步调试通用性的关键测试目标——它没有使用 Tokio 等外部异步运行时，完全依靠自定义 executor。在 embassy 上成功追踪，意味着工具不依赖任何特定运行时的内部数据结构。

³⁰embassy：一个用 Rust 编写的嵌入式异步框架，不依赖 Tokio 等外部运行时，使用自定义 executor 调度异步任务。

4.2.1 测试设置

在 embassy 示例程序上启用异步追踪，步骤如下：

1. 编译 embassy 示例程序（带调试符号），确保 ELF 文件包含完整的 DWARF 信息。
2. 在 launch.json 中启用异步调试（asyncDebug: true）并配置 GDB 连接。
3. 启动调试会话，打开 Async Inspector 面板。
4. 点击 Gen Whitelist——GDB Python 脚本扫描 embassy ELF 的符号表，生成按 crate 分组的候选函数列表。
5. 勾选用户 crate，点击 Apply Whitelist，选择目标函数点击 Trace。
6. 继续执行，程序在下一个断点处自动暂停，Async Inspector 面板刷新，展示逻辑调用树。

4.2.2 验证结果

以下为 embassy 示例程序 tick 的异步跟踪输出，展示了从最外层任务到最内层计时器的完整等待链：

```

1 ASYNC USER  tick::_embassy_main_task::_embassy_main_task_inner_function::{async_fn#0}
2   CallGraph: calls=1 | exit=1 | active=no   | thread=1
3 ASYNC USER  tick::_run_task::_run_task_inner_function::{async_fn#0}
4   CallGraph: calls=3 | exit=1 | active=yes  | thread=1
5 ASYNC USER  embassy_time::timer::{impl#5}::poll
6   CallGraph: calls=3 | exit=2 | active=yes  | thread=1

```

代码段 10: embassy tick 程序的异步跟踪输出

从追踪数据中可以观察到三个关键信息：

- 三层嵌套等待链：main_task → run_task → timer::poll，__awatee 字段按需发现了完整的依赖关系，所有节点均位于用户 crate（USER），正确过滤了框架内部的 poll 函数。
- poll 频次反映执行特征：最外层 main_task 仅被 poll 1 次即完成（active=no），内层 run_task 和 timer::poll 被 poll 3 次后仍在等待（active=yes）——符合计时器驱动的异步执行模式：计时器未到期时每次 poll 都返回 Pending，需多次重试。

- 实例级状态追踪：每个协程节点标注了 calls (poll 次数)、exit (返回次数) 和 active (是否仍在执行)，跨 poll 周期累积的状态清晰可辨。

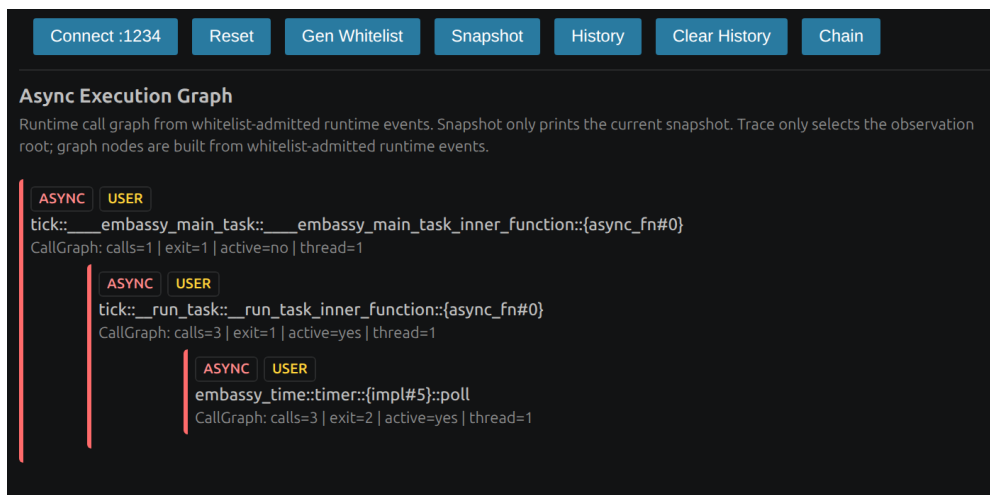


图 14: embassy tick 程序的异步函数执行图：展示了 main_task → run_task → timer::poll 的三层嵌套等待拓扑

4.2.3 验证结论

embassy 使用自定义 executor, 其内部调度结构与 Tokio 截然不同。工具能在其上正确追踪三层嵌套的等待链, 证明追踪能力仅依赖编译器生成的标准状态机结构 (``\_\_awaiter`` 字段), 不依赖任何特定运行时的私有元数据。无论是 Tokio 的工作窃取调度器、embassy 的自定义 executor, 还是操作系统的内核调度器, 只要编译器按 Rust 标准生成 async 状态机, 工具就能工作。

4.3 StarryOS 跨特权级调试验证

StarryOS 是一个基于 ArceOS 框架的组件化 Rust 操作系统, 其源码组织方式、系统调用路径、进程模型均与教学 OS 有根本差异。在 StarryOS 上验证通过, 意味着 §3.4 的三项通用化改进在组件化 OS 场景下全部生效。

4.3.1 调试配置

以下是在 StarryOS 上启用跨特权级调试的完整 launch.json 配置, 其中每个关键项均对应 §3.4 的一项改进:

```
1 {
2   "version": "0.2.0",
3   "configurations": [{
4     "type": "osdb",
5     "request": "attach",
6     "name": "StarryOS Debug (riscv64)",
7     "cwd": "${workspaceFolder}",
8     "target": ":1234",
9     "gdbpath": "/opt/riscv/bin/riscv64-unknown-elf-gdb",
10    "executable": "${workspaceFolder}/target/riscv64gc-unknown-none-elf/release
11    /starryos",
12    "qemuPath": "qemu-system-riscv64",
13    "qemuArgs": [
14      "-L", "/usr/share/qemu", "-m", "1G", "-smp", "1",
15      "-machine", "virt", "-bios", "default",
16      "-kernel", "${workspaceFolder}/StarryOS_riscv64-qemu-virt.bin",
17      "-nographic",
18      "-device", "virtio-blk-pci,drive=disk0",
19      "-drive", "id=disk0,if=none,format=raw,file=${workspaceFolder}/make/disk.
20      img",
21      "-device", "virtio-net-pci,netdev=net0",
22      "-netdev", "user,id=net0,hostfwd=tcp::5555-:5555,hostfwd=udp::5555-:5555"
23    ],
24    "first_breakpoint_group": "kernel",
25    "stopAtConnect": true,
26    "program_counter_id": 32,
27    "kernel_memory_ranges": [
28      ["0xffffffffc00000000", "0xffffffffffffffff"]
29    ],
30    "user_memory_ranges": [
```

```
30     ["0x00000000000001000", "0x0000004000000000"]
31 ],
32 "border_breakpoints": [
33     { "function": "enter_user",
34       "direction": "kernel_to_user" },
35     { "function": "starry_kernel::syscall::handle_syscall",
36       "direction": "user_to_kernel" }
37 ],
38 "hook_breakpoints": [{
39     "breakpoint": {
40         "file": "${workspaceFolder}/kernel/src/syscall/task/execve.rs",
41         "line": 65
42     },
43     "behavior": {
44         "functionArguments": "",
45         "functionBody": "const p = await this.getStringVariable('path'); const
name = p.replace('./', '').split('/').pop(); return '/home/iristseng/data/
StarryOS/user_apps/' + name + '.c';",
46         "isAsync": true
47     }
48 }],
49 "filePathToBreakpointGroupNames": {
50     "isAsync": false,
51     "functionArguments": "filePathStr",
52     "functionBody": "if (filePathStr.includes('kernel/src')) { return ['
kernel']; } else { return [filePathStr]; }"
53 },
54 "breakpointGroupNameToDebugFilePaths": {
55     "isAsync": false,
56     "functionArguments": "groupName",
57     "functionBody": "if (groupName === 'kernel') { return ['${workspaceFolder
}/target/riscv64gc-unknown-none-elf/release/starryos']; } else { return [
groupName.replace('.c', '')]; }"
```

```
58     }  
59   }]  
60 }
```

代码段 11: StarryOS 调试配置, 标注了与 §3.4 各项改进的对应关系

改进一验证——函数名断点。`enter_user` (kernel→user 方向, 位于外部 crate `axhal`, 源码不在工作区内) 和 `starry_kernel::syscall::handle_syscall` (user→kernel 方向, 使用完整的 Rust 模块路径指定) 均通过函数名指定。前者验证了跨 crate 断点定位能力, 后者验证了通用收敛点方案——一个函数名断点覆盖所有用户程序的全部系统调用返回路径。

改进一验证——方向属性。两个边界断点同处内核地址空间 (`0xffffffffc000000000` 起始), PC 寄存器值无法区分执行方向。配置中 `enter_user` 标记为 `kernel_to_user`, `handle_syscall` 标记为 `user_to_kernel`, 状态机在每次 GDB 停止时同时校验方向字段和当前状态, 确保两个同在内核空间的边界断点被正确识别。

改进二验证——跨 Rust 版本变量读取。Hook 断点的行为函数通过 `getStringVariable('path')` 读取 `execve` 的程序路径参数。该函数不依赖任何 Rust 版本特定的 String 内部字段路径——StarryOS 内核使用 Release 编译 (见配置中 `executable` 指向 `release/starryos`), 其 Rust 编译器版本可能与调试器开发环境不同, 但通过 GDB pretty printer + 协议层 console 捕获 + 内存直读的方案, 路径读取始终正确。行为函数进一步从路径中提取文件名 (`replace('.', '/', '').split('/').pop()`), 拼接为用户程序源码路径, 作为新断点组的名称。

改进三验证——动态进程组自动注入。Hook 断点打在 `execve.rs:65` (`execve` 系统调用的实现位置)。当用户通过 shell 启动任意程序时, 行为函数读取路径并映射为对应的 `.c` 源码文件路径, 触发动态进程组创建。新组通过待注入队列自动继承 `starry_kernel::syscall::handle_syscall` 边界断点, 确保 user→kernel 方向的切换链条完整闭合。

断点组路由配置。除三项改进外, 配置中的 `filePathToBreakpointGroupNames` 和 `breakpointGroupNameToDebugFilePaths` 是两个可编程的映射函数, 分别负责「文件路径 → 断点组名」和「断点组名 → 符号文件路径」的路由。前者将内核源码路径 (含 `kernel/src`) 路由到 `kernel` 组, 其他路径以自身作为组名; 后者将 `kernel` 组映射到内核 ELF, 其他组名去掉 `.c` 后缀作为用户程序的 ELF 路径。

4.3.2 调试流程验证

完整的调试流程已在 StarryOS 上验证通过，涵盖了从内核启动到用户程序动态运行的完整生命周期：

1. QEMU RISC-V 虚拟机启动（按 launch.json 配置加载内核镜像和磁盘镜像），调试器通过 GDB remote 协议连接端口 1234，加载内核符号文件，进入 kernel 断点组。
2. 内核初始化过程中，状态机在 enter_user (kernel→user 边界断点) 处自动触发——进入单步模式，逐指令执行直到 PC 进入用户地址空间，切换至用户进程断点组，继续执行。
3. StarryOS 进入交互式 shell，等待用户输入。
4. 用户在 shell 中输入命令启动用户程序（如 /test_user），内核执行 execve 系统调用。
5. Hook 断点在 execve.rs:65 命中：getStringVariable('path') 通过 GDB pretty printer + console 捕获 + 内存直读获取程序路径；行为函数从路径中提取文件名并拼接为源码路径（如 /home/.../user_apps/test_user.c）作为新进程组名称；自动创建新断点组；从待注入队列自动继承 starry_kernel::syscall::handle_syscall 边界断点；加载用户程序的符号文件；切换至新组。
6. 用户程序运行期间可正常设置源代码断点、查看变量和调用栈。当用户程序发起系统调用时，CPU 执行 ecall 陷入内核，状态机检测到栈帧函数名匹配 starry_kernel::syscall::h (user→kernel 边界断点)，走 PC 快速路径直接切换回 kernel 断点组。
7. 系统调用返回时，状态机再次触发 kernel→user 切换，回到用户进程断点组。多次系统调用 → 返回的循环中，断点组和符号文件正确切换，断点无残留无丢失。

整个调试过程中，开发者无需手动干预符号表或断点——在内核和用户程序的任意源码位置正常设置断点，调试器自动处理特权级切换。StarryOS 的完整调试流程录屏以及调试指导文档均存放于项目仓库。

4.4 Async-os 统一调试场景验证

Async-os³¹是本项目的终极验证目标——同时启用特权级切换管理和异步执行流还原，要求 §3.1 的运行时发现机制和 §3.2 的保存恢复机制在异步操作系统的真实场景下协同工作。

验证分两个阶段推进。

4.4.1 第一阶段：内核态异步协程追踪（已完成）

以 Async-os 的 `coroutine_test` 应用为目标，启用异步调试功能，验证调试器能否在内核态正确追踪异步协程。以下为调试配置：

```
1 {
2     "version": "0.2.0",
3     "configurations": [{
4         "type": "ardb",
5         "request": "attach",
6         "name": "async-os coroutine_test debug",
7         "cwd": "${workspaceFolder}",
8         "target": ":1234",
9         "gdbpath": "gdb-multiarch",
10        "executable": "${workspaceFolder}/apps/coroutine_test/
coroutine_test_riscv64-qemu-virt.elf",
11        "qemuPath": "qemu-system-riscv64",
12        "qemuArgs": [
13            "-m", "2G", "-smp", "1",
14            "-machine", "virt", "-bios", "default",
15            "-kernel", "${workspaceFolder}/apps/coroutine_test/
coroutine_test_riscv64-qemu-virt.bin",
16            "-nographic",
17            "-s", "-S"
18        ],
19        "first_breakpoint_group": "kernel",
```

³¹Async-os：一个用 Rust 编写的异步微内核操作系统，内核调度器和 I/O 处理均基于 `async/await` 机制实现，是验证统一调试平台终极目标的理想场景。

```
20     "stopAtConnect": true,  
21     "program_counter_id": 32,  
22     "kernel_memory_ranges": [  
23         ["0xffffffffc00000000", "0xffffffffffffffff"]  
24     ],  
25     "border_breakpoints": [],  
26     "hook_breakpoints": []  
27 }]  
28 }
```

代码段 12: Async-os coroutine_test 的内核态异步追踪配置

此阶段仅验证内核态的异步追踪，尚未涉及特权级切换，因此 `border_breakpoints` 和 `hook_breakpoints` 均为空。调试器可以成功获取内核态异步协程的逻辑调用树，Async Inspector 面板正确展示了协程节点及其等待关系。

4.4.2 第二阶段：跨特权级异步追踪（进行中）

在第一阶段的基础上，以 `user_boot` 应用为目标，同时启用 `osDebug` 和 `asyncDebug`，尝试在内核态与用户态之间切换的同时维护异步追踪链。

目前已完成了调试配置文件的撰写，调试器能够在特权级切换时正确触发断点组切换。

```
1  {  
2      "version": "0.2.0",  
3      "configurations": [  
4          {  
5              "type": "ardb",  
6              "request": "attach",  
7              "name": "async-os (原版) debug",  
8              "cwd": "${workspaceFolder}",  
9              "target": ":1234",  
10             "gdbpath": "gdb-multiarch",  
11             "executable": "${workspaceFolder}/apps/user_boot/user_boot_riscv64-qemu-virt.elf",
```

```

12     "qemuPath": "qemu-system-riscv64",
13     // "debugServer": "localhost:4711",    // 调试调试器时启动
14     "qemuArgs": [
15         "-m", "2G",
16         "-smp", "1",
17         "-machine", "virt",
18         "-bios", "default",
19         "-kernel", "${workspaceFolder}/apps/user_boot/user_boot_riscv64
-qemu-virt.bin",
20         "-device", "virtio-blk-pci,drive=disk0",
21         "-drive", "id=disk0,if=none,format=raw,file=${workspaceFolder}/
disk.img",
22         "-device", "virtio-net-pci,netdev=net0",
23         "-netdev", "user,id=net0",
24         "-nographic",
25         "-s", "-S"
26     ],
27     "first_breakpoint_group": "kernel",
28     "second_breakpoint_group": "pipetest",
29     "stopAtConnect": true,
30     "program_counter_id": 32,
31     "kernel_memory_ranges": [
32         ["0xffffffffc000000000", "0xffffffffffffffff"]
33     ],
34     "user_memory_ranges": [
35         ["0x0000000000000000", "0x0000004000000000"]
36     ],
37     "border_breakpoints": [
38         { "function": "<taskctx::arch::riscv::TrapFrame>::user_return"
,"direction": "kernel_to_user"},
39         { "function": "trampoline::task_api::user_task_top::{async_fn
#0}" ,"direction": "user_to_kernel"}
40     ],

```

```

41     "hook_breakpoints": [
42         {
43             "breakpoint": { "function": "syscall::syscall_task::imp::
task::syscall_exec::{async_fn#0}" },
44             "behavior": {
45                 "functionArguments": "",
46                 "functionBody": "try { const p = await this.
getStringVariable('((const char**)(args[0]))'); return p ? p.split('/').pop
() || 'user' : 'user'; } catch(e) { return 'user'; }",
47                 "isAsync": true
48             }
49         }
50     ],
51     "filePathToBreakpointGroupNames": {
52         "functionArguments": "filepath",
53         "functionBody": "if (filepath.includes('modules/')) || filepath.
includes('crates/')) || filepath.includes('apps/')) return ['kernel']; if (
filepath.includes('user_apps/')) || filepath.includes('user-lib/')) ||
filepath.includes('uruntime/')) return ['user']; return ['kernel'];",
54         "isAsync": false
55     },
56     "breakpointGroupNameToDebugFilePaths": {
57         "functionArguments": "groupName",
58         "functionBody": "if (groupName === 'kernel') return []; const
fs = require('fs'); const path = require('path'); const dir = 'user_apps/
target/riscv64gc-unknown-linux-musl/release/'; if (fs.existsSync(dir)) {
const files = fs.readdirSync(dir).filter(f => f.includes(groupName) && !fs.
statSync(path.join(dir, f)).isDirectory()); if (files.length > 0) return [{
path: path.join(dir, files[0]), textAddr: '0x4000000'}]; } return [{path:
path.join(dir, groupName), textAddr: '0x4000000'}];",
59         "isAsync": false
60     }
61 }

```

62]
63 }

目前的调试效果如下，但跨特权级的异步追踪链尚未完全打通——调试器在同时启用 osDebug 和 asyncDebug 时存在逻辑 Bug，正在排查修复中。:

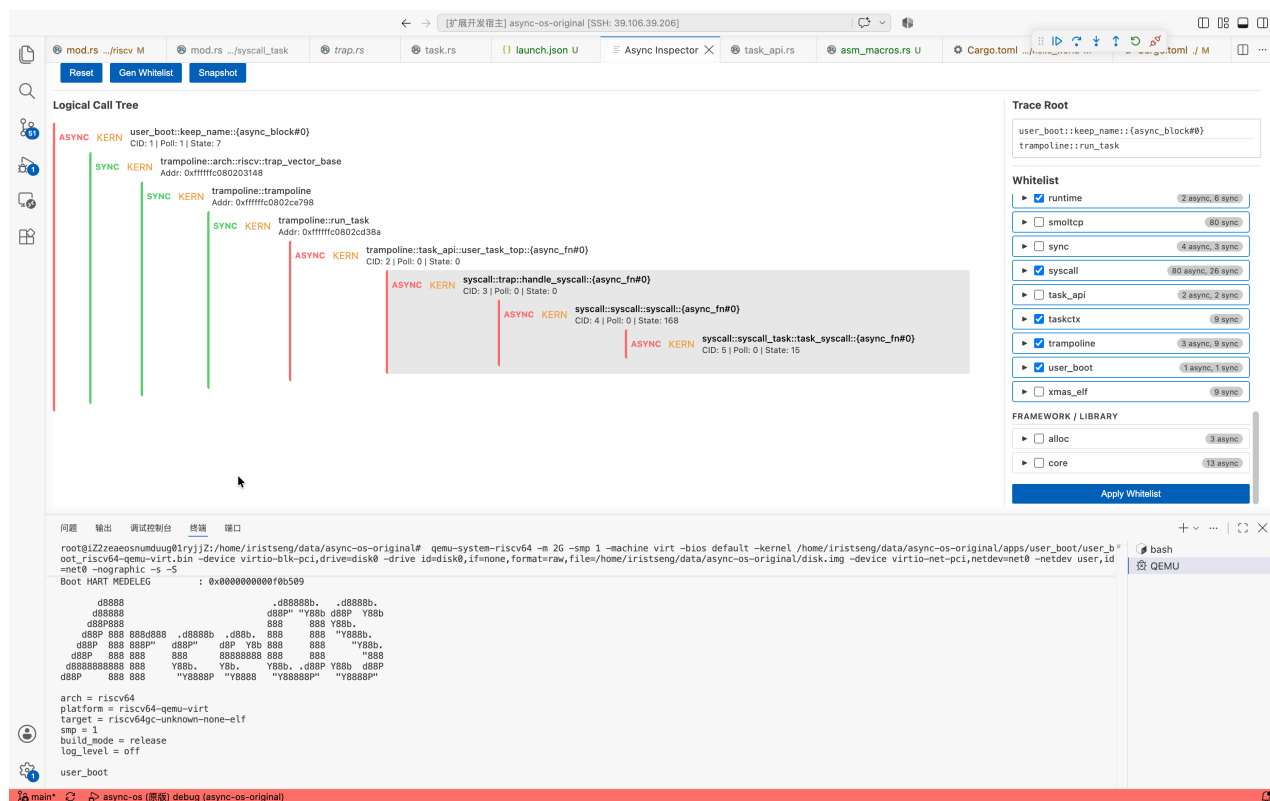


图 15: Async-os 跨特权级异步跟踪

跨特权级异步追踪是统一调试平台的终极场景，当前工作仍在推进中。

5 总结与展望

5.1 已完成工作总结

本项目围绕四个核心问题，构建了一个跨特权级的统一调试平台：

1. 异步跟踪方案改进。将等待关系获取从「调试前离线解析 DWARF 预计算依赖树并全量插桩」改为「运行时通过 __awaitee 字段按需动态发现」，引入 (poll_symbol,

env_ptr) 实例级识别和 trace root 概念, 将追踪开销从与程序规模成正比降低为与当前执行路径长度成正比。

2. 异步跟踪与跨特权级调试融合。设计跨生命周期的 save/restore 机制, 维护断点数据、协程状态、追踪记录和白名单配置四类共 19 项数据在特权级切换前后的连续性, 实现异步操作系统调试。
3. 图形化界面设计。实现 VS Code 插件化三层架构, Async Inspector 面板以交互式树形图承载等待拓扑、协程元数据和节点身份三类异步语义, 与原生 Call Stack 形成双轨协同。
4. 调试器通用性改进。通过函数名断点、通用收敛点 + 方向属性、动态进程组自动注入、跨 Rust 版本变量读取等技术, 使调试器摆脱对教学 OS 静态特征的依赖, 覆盖组件化 OS 等更广泛的目标。

在验证方面, 37 个状态机单元测试和 4 个集成测试全部通过; embassy 上验证了运行时无关性 (三层嵌套等待链正确追踪); StarryOS 上完成了从内核初始化到动态用户程序启动的全流程调试; Async-os 上完成了内核态异步协程追踪, 跨特权级异步追踪仍在推进中。

5.2 未来展望

统一调试平台的核心能力已经建立, 但在易用性和性能方面仍有提升空间。下一步工作包括:

1. 调试配置的自动推导。当前配置中的地址范围、syscall handler 函数名、内核出口函数名仍需开发者手动指定。计划通过解析内核 ELF 符号表和链接脚本, 自动推导这些配置项, 进一步提升调试方案的通用性。
2. 特权级切换性能优化。当前 kernel→user 方向的边界断点命中后, 状态机进入逐指令单步模式穿过整个切换函数体 (约 30 条指令)。在 shell 交互场景中, 用户每输入一个字符可能触发数次完整的特权级切换, 单字符延迟可达秒级。优化方向包括使用 GDB finish 命令替代逐指令单步和引入切换冷却期机制。

6 项目分工

成员	主要分工
曾小红	异步跟踪方案改进；异步跟踪与 OS 调试融合；Async Inspector 图形化面板设计与实现；StarryOS 适配与验证；embassy 异步追踪验证；Async-os 适配探索；撰写参赛文档；参赛 PPT 制作
王浩铭	OS 调试功能测试与验证（状态机单元测试、集成测试）；硬件调试
武雪妍	文档图片与示意图制作；参赛 PPT 制作

表 2: 项目分工